A consistency proof for BGA carried out inside grounded arithmetic in Isabelle/Pure

Omar Muhammad

October 13, 2026

Abstract

Grounded arithmetic admits arbitrary recursive definitions by weakening inference rules with *habeas quid* termination premises. The existing mechanized consistency results for grounded arithmetic are carried out in Isabelle/HOL, that is, in a classical metatheory far stronger than the object logic. This report documents a development that carries the argument out inside a version of grounded arithmetic itself. QGA, an axiomatization of grounded arithmetic with quantifiers, is embedded into Isabelle/Pure in `GD.thy`. Basic grounded arithmetic BGA is arithmetized on top of it in `BGA_on_GA.thy`: terms, formulas, judgments, proofs, the operational semantics and the proof checker are encoded as QGA natural numbers, and the checker is proved sound with respect to the encoded semantics under a small set of abstract assumptions about the encoding. A concrete Cantor-pairing encoding in `encode.thy` discharges those assumptions and yields, as a theorem of QGA: for any two numbers coding BGA terms a and b, no numbers p_1 and p_2 code BGA proofs, from empty hypotheses, of $a = b$ and of $a \neq b$ respectively.

Contents

1 Overview **2**
1.1 The three layers . . . 2
1.2 What is proved . . . 3
1.3 The two levels . . . 3

2 QGA **4**
2.1 Formulas and terms . . . 4
2.2 Structural rules . . . 4
2.3 Propositional rules . . . 5
2.4 Equality and the habeas quid judgments . . . 6
2.5 Natural numbers . . . 7
2.6 Quantifiers . . . 8
2.7 Conditionals . . . 9
2.8 The definitional mechanism . . . 9
2.9 Arithmetic . . . 10
2.10 Induction principles . . . 10
2.11 Pairing . . . 11
2.12 Lists . . . 12
2.13 Summary . . . 13

3 BGA arithmetized in QGA **13**
3.1 The system being encoded . . . 13
3.2 Codes . . . 13
3.3 The locale tower . . . 15
3.4 The consistency skeleton . . . 16
3.5 Encoded syntax . . . 17
3.6 `bga_encoding` . . . 18
3.7 `bga_dfns` . . . 20
3.8 Semantics . . . 20
3.9 Satisfaction at a fixed budget . . . 22
3.10 `bga_subst` and `asn_put` . . . 23
3.11 The proof rules of BGA . . . 24
3.12 `bga_subst_rule` . . . 24
3.13 `bga_eq_rule` . . . 26
3.14 `bga_ind_rule` . . . 27
3.15 `bga_struct_rule` . . . 28
3.16 `bga_app_rule` . . . 29
3.17 `bga_proof_check` . . . 30

4 The consistency proof **31**
4.1 Shape of the argument . . . 31
4.2 The evaluator is a partial function . . . 32
4.3 Soundness of a single step . . . 32
4.4 From steps to proofs . . . 34
4.5 Consistency . . . 35

5 The concrete encoding **36**
5.1 The encoders . . . 37
5.2 Concrete definitions . . . 37
5.3 Discharging the assumptions . . . 38
5.4 Instantiation and the theorem . . . 40

6 Reduction of QGA to BGA **41**

7 Conclusion **41**

A An unfinished attempt at QGA self-consistency **42**
A.1 The proposed semantics . . . 42
A.2 Ideas retained from the attempt . . . 42

1 Overview

1.1 The three layers

The development consists of three theories, each importing the previous one.

Theory	Imports	Contributes
`GD.thy`	`Pure`	QGA, an embedding of grounded arithmetic in Isabelle/Pure
`BGA_on_GA.thy`	`GD`	BGA arithmetized in QGA, and the soundness proof for its proof checker
`encode.thy`	`BGA_on_GA`	a concrete encoding discharging every assumption, and the consistency theorem

`BGA_on_GA.thy` does the heavy lifting. It never commits to a particular Gödel numbering. It fixes an interface (`tag_T`, `load_T`, `pack_T` and their formula counterparts) together with the assumptions used by the soundness argument, and proves everything relative to that interface. `encode.thy` then pays the arithmetic cost once, by exhibiting one encoding that satisfies the assumptions, and instantiates the result.

1.2 What is proved

The final theorem, `BGA_syntactically_consistent`, says that for any two numbers coding BGA terms a and b, no numbers p_1 and p_2 code BGA proofs, from empty hypotheses, of $a = b$ and of $a \neq b$ respectively.

The result establishes the consistency of the encoded BGA proof system in QGA as implemented over Pure, with the recursive equations used by the encoding. The development does not import Isabelle/HOL. It is not a self-consistency proof: obtaining one for BGA by this route would require a suitable reduction of the QGA reasoning used here to BGA, together with an internal justification of the required proof translation. Reflective grounded arithmetic (RGA) is a related system introduced by Ford [2]. Subsequent work has shown that RGA cannot prove its own consistency; that work has not yet been published. Whether an analogous obstruction applies to the present QGA system remains to be established.

1.3 The two levels

QGA is implemented as an extension of Isabelle/Pure: `GD.thy` imports `Pure` and adds the types, constants and axioms of QGA. The distinction between two levels is between QGA formulas and the Pure judgments used to state and apply its rules. In this development, we use QGA to prove the consistency of the encoded BGA proof system.

QGA is the logic in which we write the proof. Its formulas are the terms of type `o`. Its vocabulary, given in full in §2, consists of the primitives `∨` and `¬`, the connectives `∧`, `⟶` and `⟷` defined from them, the equality `=`, and the quantifiers `∀` and `∃`.

Isabelle/Pure is the framework in which QGA is written down. Pure is a small calculus whose purpose is to state the rules of other logics, and it has three symbols. `A ⟹ B` says that from a derivation of A one obtains a derivation of B, so a rule "from A infer B" is written this way. `⋀x. A` says that A was derived for an arbitrary x, which is how the eigenvariable condition of a rule is expressed: the premise `⋀x. x N ⟹ Q x` of `forallI` asks us to prove `Q x` for arbitrary `x`, assuming `x N`. The other standing assumptions must not mention `x`. `a ≡ b` is the definitional equality of the framework, used where a QGA notion is abbreviated rather than axiomatized.[1]

The type system keeps the two levels apart. `o` is the type of QGA formulas and `prop` the type of Pure judgments, connected by `Trueprop :: o ⇒ prop`, which is declared with the `judgment` command and printed invisibly. Nothing maps `prop` back to `o`. A QGA formula can therefore be asserted as a judgment, while `⟹` and `⋀` can never appear inside a QGA formula.

[1] Definitional equality identifies terms that differ only in ways the logic cannot see: renaming a bound variable (α), applying a lambda to its argument (β), and replacing `λx. f x` by `f` (η). Unfolding a `definition` is also an `≡` step.

Pure holds hypothetical judgments freely, because $\Longrightarrow$ may always be introduced. Converting one into a QGA formula requires `implI`, whose premise `a B` demands that the antecedent be decided. *Habeas quid* is a gate between the two levels, and every implication occurring in a QGA formula has passed through it.

This arrangement is what supports the claim that the development is carried out in QGA using Isabelle/Pure as its framework. Pure's kernel offers assumption, introduction and elimination for $\Longrightarrow$ and $\bigwedge$, the rules for $\equiv$, instantiation of schematic variables, and citation of an axiom. Only the last of these introduces an assertion. The others move an assertion between hypothesis and antecedent position, replace a rule schema by one of its instances, or work on equalities. Remove the axioms of `GD.thy` and Pure proves no assertion at all except from an assumption of that same assertion, since to Pure the QGA connectives are constants about which it knows nothing. Every QGA formula established in the development therefore comes from a QGA rule.

One kernel rule needs care. From `A ≡ B` and `A` one may infer `B`, so an equality can introduce an assertion that appears nowhere in its premises. Inside Pure, equalities come only from the conversions described in the footnote, none of which changes meaning. They can also come from an axiom, and two axioms of `GD.thy` are of that kind: `eq_reflection` (`x = y ⟹ x ≡ y`) and `iff_reflection` (`p ⟷ q ⟹ p ≡ q`), which exist so that Isabelle's simplifier can act as a derived rule. These two are the entire surface through which the framework could contribute strength. Each mirrors a QGA rule together with congruence, `eq_reflection` mirroring `eqSubst`, whose context `Q :: num ⇒ o` is already arbitrary, so both are expected to be pretty tame.

A last reliance on the framework concerns the quantifier syntax. QGA formulas are the terms of type `o`, and `forall` has type `(num ⇒ o) ⇒ o`, so a quantified formula is `forall` applied to a Pure function. Binding is therefore the framework's, and renaming, capture-avoiding substitution and freshness come for free rather than being axiomatized. This is the usual higher-order abstract syntax treatment of a binder and is what keeps §2 free of a substitution calculus.

In the natural-deduction presentations below, QGA rules are written with premises above the line and the conclusion below, with no explicit context. A premise discharged by a rule is written in brackets above the derivation that uses it. BGA judgments carry an explicit list of hypotheses, so BGA rules are written as sequents $\Gamma \vdash \varphi$. All Isabelle snippets are extracted verbatim from the theories by `extract.sh`, which replaces Isabelle's ASCII escapes by the corresponding Unicode glyphs and changes nothing else.

2 QGA

`GD.thy` axiomatizes grounded arithmetic as an object logic over Isabelle/Pure. The propositional and arithmetic core, and the definitional mechanism are from Kehrli's development, who established that grounded arithmetic can be axiomatized in Pure and reasoned in productively [1].

2.1 Formulas and terms

Two Pure types carry the embedding. `o` is the type of QGA formulas and `num` is the type of QGA terms. A `judgment` declaration supplies the coercion `Trueprop :: o ⇒ prop` that lets a formula be asserted at the framework level, so that a QGA formula can appear as the conclusion of a Pure rule.

2.2 Structural rules

The rules in the rest of this section are written without a hypothesis context, because Pure maintains one. Making it visible for a moment, a QGA judgment is $\Gamma \vdash \varphi$ with Γ a finite set of

formulas, and the framework supplies the operations that move judgments around.

$$\frac{}{\Gamma, \varphi \vdash \varphi}\ (\textsc{hyp}) \qquad \frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi}\ (\textsc{weak}) \qquad \frac{\Gamma \vdash \psi \quad \Gamma, \psi \vdash \varphi}{\Gamma \vdash \varphi}\ (\textsc{cut})$$

$$\frac{\Gamma \vdash \varphi}{[x \mapsto t]\Gamma \vdash [x \mapsto t]\varphi}\ (\textsc{inst})$$

None of these is an axiom of `GD.thy`. HYP is Pure's `assume`, WEAK and CUT are introduction and elimination of $\Longrightarrow$, and INST is instantiation of a schematic variable. A rule written `⟦P; Q⟧ ⟹ R` therefore means the judgment $P, Q \vdash R$, and a theorem written that way holds for every instance of its free variables. Contraction and exchange do not appear at all, since Γ is a set.

One further operation has no sequent form. If $\Gamma \vdash \varphi$ is derived for a variable x that occurs nowhere in Γ, the derivation is uniform in x, and Pure records this as `⋀x`. This is the eigenvariable condition, and it is what the premises of IND and ∀I below rely on.

2.3 Propositional rules

Disjunction and negation are primitive.

$$\frac{P}{P \vee Q}\ (\vee\text{I1}) \qquad \frac{Q}{P \vee Q}\ (\vee\text{I2}) \qquad \frac{\neg P \quad \neg Q}{\neg(P \vee Q)}\ (\vee\text{I3})$$

$$\frac{\neg(P \vee Q)}{\neg P}\ (\vee\text{E2}) \qquad \frac{\neg(P \vee Q)}{\neg Q}\ (\vee\text{E3}) \qquad \frac{P \vee Q \qquad \begin{matrix}[P] \\ \vdots \\ R\end{matrix} \qquad \begin{matrix}[Q] \\ \vdots \\ R\end{matrix}}{R}\ (\vee\text{E1})$$

$$\frac{P}{\neg\neg P}\ (\neg\neg\text{I}) \qquad \frac{\neg\neg P}{P}\ (\neg\neg\text{E}) \qquad \frac{P \quad \neg P}{Q}\ (\textsc{exf})$$

```
typedecl o

judgment
  Trueprop :: ‹o ⇒ prop›  (‹_› 5)

axiomatization
  disj :: ‹o ⇒ o ⇒ o›  (infixr ‹∨› 30) and
  not :: ‹o ⇒ o› (‹¬ _› [40] 40)where
  disjI1: ‹P ⟹ P ∨ Q› and
  disjI2: ‹Q ⟹ P ∨ Q› and
  disjI3: ‹⟦¬P; ¬Q⟧ ⟹ ¬(P ∨ Q)› and
  disjE1: ‹⟦P ∨ Q; P ⟹ R; Q ⟹ R⟧ ⟹ R› and
  disjE2: ‹¬(P ∨ Q) ⟹ ¬P› and
  disjE3: ‹¬(P ∨ Q) ⟹ ¬Q› and
  dNegI: ‹P ⟹ (¬¬P)› and
  dNegE: ‹(¬¬P) ⟹ P› and
  exF: ‹⟦P; ¬P⟧ ⟹ Q›
```

The introduction rules are the classical ones and the De Morgan directions ∨E2 and ∨E3 are available unconditionally. What is weakened is case analysis. ∨E1 needs an actual proof of $P \vee Q$, and $P \vee Q$ is not derivable for arbitrary P: the law of excluded middle is a judgment to be established rather than an axiom, and it is written `P B`.

2.4 Equality and the habeas quid judgments

Equality is a binary predicate on terms, `num ⇒ num ⇒ o`, axiomatized by substitution and symmetry.

$$\frac{a = b \quad Q\,a}{Q\,b}\ (=\text{E}) \qquad \frac{a = b}{b = a}\ (=\text{S})$$

```
  eq :: ‹num ⇒ num ⇒ o›  (infixl ‹=› 45)
where
  eqSubst: ‹⟦a = b; Q a⟧ ⟹ Q b› and
  eqSym: ‹a = b ⟹ b = a› and
  eq_reflection: ‹x = y ⟹ x ≡ y›
```

Reflexivity is absent, and this is the pivot of the whole system. Transitivity follows from one substitution, `eq_trans: a = b ⟹ b = c ⟹ a = c`. The third axiom, `eq_reflection`, lifts a derivable equation to the framework equality `≡`, which is what lets Isabelle's simplifier operate on QGA goals; its relationship to object-level substitution is discussed in §1.3, where the remaining adequacy obligations are distinguished from the implemented axioms.

The two habeas quid judgments and the remaining connectives are definitions.

```
definition neq :: ‹num ⇒ num ⇒ o› (infixl ‹≠› 45)
  where ‹a ≠ b ≡ ¬ (a = b)›
definition bJudg :: ‹o ⇒ o› (‹_ B› [21] 20)
  where ‹(p B) ≡ (p ∨ ¬p)›
definition isNat :: ‹num ⇒ o› (‹_ N› [21] 20)
where "x N ≡ x = x"
definition conj :: ‹o ⇒ o ⇒ o› (infixl ‹∧› 35)
  where ‹p ∧ q ≡ ¬(¬p ∨ ¬q)›
definition impl :: ‹o ⇒ o ⇒ o› (infixr ‹⟶› 25)
  where ‹p ⟶ q ≡ ¬p ∨ q›
definition iff :: ‹o ⇒ o ⇒ o› (infixl ‹⟷› 25)
  where ‹p ⟷ q ≡ (p ⟶ q) ∧ (q ⟶ p)›
```

Definition 2.1 (Habeas quid)**.** `a N`, defined as `a = a`, asserts that the term `a` denotes a natural number, equivalently that the computation it names terminates. `p B`, defined as `p ∨ ¬p`, asserts that the formula `p` is grounded.

Since `a N` unfolds to `a = a`, reflexivity of equality says exactly that every term terminates, which is why it cannot be an axiom of a logic with unrestricted recursion. Conjunction, implication and the biconditional are defined from $\vee$ and $\neg$ in the classical way, and their rules are then derived. Conjunction comes out unconditional, while implication and the biconditional acquire habeas quid premises, because their derivations pass through $\vee$E1.

$$\frac{p \quad q}{p \wedge q}\ (\wedge\text{I}) \qquad \frac{p \wedge q}{p}\ (\wedge\text{E1}) \qquad \frac{p \wedge q}{q}\ (\wedge\text{E2})$$

$$\frac{\begin{array}{c}a\,\mathsf{B} \quad \overset{[a]}{\vdots} \\ b\end{array}}{a \longrightarrow b}\ (\longrightarrow\text{I}) \qquad \frac{a \longrightarrow b \quad a}{b}\ (\longrightarrow\text{E}) \qquad \frac{\begin{array}{cc}a\,\mathsf{B} \quad b\,\mathsf{B} & \overset{[a]}{\vdots} \quad \overset{[b]}{\vdots} \\ & b \qquad\quad a\end{array}}{a \longleftrightarrow b}\ (\longleftrightarrow\text{I})$$

Two further derived rules are used constantly. Case analysis on a grounded formula,

```
cases_bool: ⟦q B; q ⟹ p; ¬q ⟹ p⟧ ⟹ p,
```

and grounded proof by contradiction,

```
grounded_contradiction: ⟦p B; ¬p ⟹ q; ¬p ⟹ ¬q⟧ ⟹ p,    contradiction: ⟦p B; ¬p ⟹
                                                    False⟧ ⟹ p.
```

Both require the formula in question to be grounded. The final consistency argument of §4.5 is an application of `contradiction`, and the groundedness premise it needs is the reason the abstract development assumes that proof checking is boolean.

2.5 Natural numbers

$$\frac{}{0\,\mathsf{N}}\ (0\mathrm{I}) \qquad \frac{\mathsf{S}(a) = \mathsf{S}(b)}{a = b}\ (\text{S-INJ}) \qquad \frac{a = b}{\mathsf{S}(a) = \mathsf{S}(b)}\ (\text{S-CONG})$$

$$\frac{a = b}{\mathsf{P}(a) = \mathsf{P}(b)}\ (\text{P-CONG}) \qquad \frac{a\,\mathsf{N} \quad b\,\mathsf{N}}{(a = b)\,\mathsf{B}}\ (\text{EQBOOL}) \qquad \frac{a\,\mathsf{N}}{\mathsf{S}(a) \neq 0}\ (\text{S}{\neq}0)$$

$$\frac{a\,\mathsf{N}}{\mathsf{P}(\mathsf{S}(a)) = a}\ (\text{PS}) \qquad \frac{}{\mathsf{P}(0) = 0}\ (\text{P0}) \qquad \frac{(a = b)\,\mathsf{B}}{a\,\mathsf{N} \wedge b\,\mathsf{N}}\ (\text{EQE})$$

$$\frac{\mathsf{P}(a)\,\mathsf{N}}{a\,\mathsf{N}}\ (\text{PREDTI}) \qquad \frac{a\,\mathsf{N} \quad Q\,0 \qquad \begin{array}{c}[x\,\mathsf{N},\ Q\,x]\\ \vdots\\ Q\,\mathsf{S}(x)\end{array}}{Q\,a}\ (\text{IND})$$

```
  zero :: ‹num›                          and
  suc :: ‹num ⇒ num›      (‹S(_)› [800]) and
  pred :: ‹num ⇒ num›     (‹P(_)› [800])
where
  nat0: ‹zero N› and
  sucInj: ‹S a = S b ⟹ a = b› and
  sucCong: ‹a = b ⟹ S a = S b› and
  predCong: ‹a = b ⟹ P a = P b› and
  eqBool: ‹⟦a N; b N⟧ ⟹ (a = b) B› and
(*eqBoolB: ‹⟦x B; y B⟧ ⟹ (x = y) B› and*)
  sucNonZero: ‹a N ⟹ S a ≠ zero› and
  predSucInv: ‹a N ⟹ P(S(a)) = a› and
  pred0: ‹P(zero) = zero› and
  eqE: ‹((a = b) B) ⟹ ((a N) ∧ (b N))› and
  predTIE: ‹(P a N) ⟹ (a N)› and
  ind [case_names HQ Base Step]:
           "⟦a N; Q zero; ⋀x. x N ⟹ Q x ⟹ Q S(x)⟧ ⟹ Q a"
```

`eqBool` is the rule that converts termination into decidability: two terminating terms are either equal or not. `eqE` and `predTIE` run in the opposite direction, recovering termination from groundedness of an equation and from termination of a predecessor. Induction carries a habeas quid premise on the subject of the induction, and its step case may assume both `x N` and the induction hypothesis.

Truth and falsity are definitions.

```
definition True
  where ‹True ≡ zero = zero›
definition False
  where ‹False ≡ S(zero) = zero›
```

Derived facts of constant use are `zeroRefl: zero = zero`, `natS: a N ⟹ S a N`, `natSI: S a N ⟹ a N`, `natP: a N ⟹ P a N`, and the pair `eq_impl_term: a = b ⟹ a N` and `eq_impl_term2: a = b ⟹ b N`, by which a derivable equation witnesses the termination of both of its sides.

2.6 Quantifiers

$$\frac{\begin{array}{c}[x\,\mathsf{N}]\\ \vdots\\ Q\,x\end{array}}{\forall x.\,Q\,x}\ (\forall\mathrm{I}) \qquad \frac{\forall x.\,Q\,x \quad a\,\mathsf{N}}{Q\,a}\ (\forall\mathrm{E})$$

$$\frac{a\,\mathsf{N} \quad Q\,a}{\exists x.\,Q\,x}\ (\exists\mathrm{I}) \qquad \frac{\exists x.\,Q\,x \qquad \begin{array}{c}[a\,\mathsf{N},\ Q\,a]\\ \vdots\\ R\end{array}}{R}\ (\exists\mathrm{E})$$

```
  forall :: "(num ⇒ o) ⇒ o"  (binder "∀" [8] 9) and
  exists :: "(num ⇒ o) ⇒ o"  (binder "∃" [8] 9)
where
  forallI: "⟦⋀x. x N ⟹ Q x⟧ ⟹ ∀x. Q x" and
  forallE: "⟦∀c'. Q c'; a N⟧ ⟹ Q a" and
  existsI: "⟦a N; Q a⟧ ⟹ ∃x. Q x" and
  existsE: "⟦∃i. Q i; ⋀a. a N ⟹ Q a ⟹ R⟧ ⟹ R"
```

Both quantifiers range over `num` and are relativized to terminating witnesses. ∀I proves the body for an arbitrary terminating eigenvariable, ∀E instantiates only at terminating terms, and ∃I must exhibit a terminating witness.

Under these four rules a negated universal is inert. `forall` is a primitive constant, so no elimination rule matches $\neg(\forall x.\,Q\,x)$: the De Morgan rules ∨E2 and ∨E3 want a negated disjunction, ¬¬E wants a double negation, and ExF would first need $\forall x.\,Q\,x$ itself, which ∀I supplies only from a uniform derivation. Nor does any rule conclude $\neg(\forall x.\,Q\,x)$ from a counterexample. Two further axioms supply both directions.

$$\frac{a\,\mathsf{N} \quad \neg(Q\,a)}{\neg(\forall x.\,Q\,x)}\ (\neg\forall\mathrm{I}) \qquad \frac{\neg(\forall x.\,Q\,x) \qquad \begin{array}{c}[a\,\mathsf{N},\ \neg(Q\,a)]\\ \vdots\\ R\end{array}}{R}\ (\neg\forall\mathrm{E})$$

```
axiomatization where
  notForallI: "⟦a N; ¬(F a)⟧ ⟹ ¬(∀x. F x)" and
  notForallE: "⟦¬(∀x. F x); ⋀a. a N ⟹ ¬(F a) ⟹ R⟧ ⟹ R"
```

¬∀I refutes a universal from one terminating counterexample and ¬∀E extracts one, so together they restore the duality between ¬∀ and ∃¬ as a pair of inference rules. Each carries a habeas quid premise on the witness, which keeps them in line with the rest of the system: the counterexample has to be an actual number rather than a description of one. §A.1 gives the semantic clause that justifies them. They correspond to the rules taken as primitive in reflective grounded arithmetic, which are primitive there for the same reason.

The dual pair for the existential needs no axiom. From $\neg(\exists x.\,Q\,x)$, together with groundedness of Q at the point under consideration, ∃I manufactures the positive partner and ExF closes the case. This gives

```
existsNeg: ⟦¬(∃x. Q x); ⋀x. x N ⟹ (Q x) B⟧ ⟹ ∀x. ¬(Q x)
```

as a derived rule. The asymmetry is the one just described. $\exists$ has an introduction rule that applies to a local assumption, and $\forall$ does not.

Where the new axioms are used is worth recording, since it is a single place. §4 needs to decide whether a list of hypotheses is satisfied, and the negative case of that decision is the refutation of a universal. Nothing else in the development appeals to either rule.

2.7 Conditionals

The if-zero-else construct is written `if c then a else b` and is polymorphic in the branches, so it serves both for term-valued and for formula-valued conditionals.

$$\frac{c \quad a\,\mathsf{N}}{(\text{if } c \text{ then } a \text{ else } b) = a}\ (\textsc{condI1}) \qquad \frac{\neg c \quad b\,\mathsf{N}}{(\text{if } c \text{ then } a \text{ else } b) = b}\ (\textsc{condI2})$$

$$\frac{c \quad (\text{if } c \text{ then } a \text{ else } b)\,\mathsf{N}}{a\,\mathsf{N}}\ (\textsc{condE1}) \qquad \frac{(\text{if } c \text{ then } a \text{ else } b)\,\mathsf{N}}{c\,\mathsf{B}}\ (\textsc{condE3})$$

$$\frac{c \quad Q\,(\text{if } c \text{ then } a \text{ else } b)}{Q\,a}\ (\textsc{cond-then-E}) \qquad \frac{c \quad Q\,a}{Q\,(\text{if } c \text{ then } a \text{ else } b)}\ (\textsc{cond-then-I})$$

```
  cond :: ‹o ⇒ 'a ⇒ 'a ⇒ 'a› (‹if _ then _ else _› [25, 24, 24] 24)

axiomatization where
  condI1: ‹⟦c; a N⟧ ⟹ (if c then a else b) = a› and
  condI2: ‹⟦¬c; b N⟧ ⟹ (if c then a else b) = b› and
  (*condT: ‹⟦c B; a N; b N⟧ ⟹ if c then a else b N› and*)
  condI1B: ‹⟦c; d B⟧ ⟹ (if c then d else e) ⟷ d› and
  condI2B: ‹⟦¬c; e B⟧ ⟹ (if c then d else e) ⟷ e› and
  (*condTB: ‹⟦c B; d B; e B⟧ ⟹ if c then d else e B›*)
  (*New additions*)
  condE1: ‹⟦c;  (if c then a else b) N⟧ ⟹ (a N)› and
  condE2: ‹⟦¬c;  (if c then a else b) N⟧ ⟹ (b N)› and
  condE3: ‹⟦(if c then a else b) N⟧ ⟹ (c B)› and
  condE1B: ‹⟦c;  (if c then d else e) B⟧ ⟹ (d B)› and
  condE2B: ‹⟦¬c;  (if c then d else e) B⟧ ⟹ (e B)› and
  condE3B: ‹⟦(if c then d else e) B⟧ ⟹ (c B)› and
  (* lazy conditional rules *)
  cond_thenQ_E: "c ⟹ Q (if c then a else b) ⟹ Q a" and
  cond_thenQ_I: "c ⟹ Q a ⟹ Q (if c then a else b)" and
  cond_elseQ_E: "¬ c ⟹ Q (if c then a else b) ⟹ Q b" and
  cond_elseQ_I: "¬ c ⟹ Q b ⟹ Q (if c then a else b)"
```

Each rule has a formula-level counterpart in which = is replaced by $\longleftrightarrow$ and N by B, and each "then" rule has an "else" twin. The last pair is the lazy form: once the guard is decided, an arbitrary context `Q` transports across the conditional. These are the rules that let a recursive definition be unfolded one branch at a time without evaluating the other branch. The two totality lemmas

```
condT: ⟦c B; a N; b N⟧ ⟹ (if c then a else b) N,    condTB: ⟦c B; a B; b B⟧ ⟹ (if c
                                                      then a else b) B
```

are derived rather than assumed, and they carry most of the groundedness proofs about the encoded checkers of §3.

2.8 The definitional mechanism

A recursive definition is admitted with no termination side condition. The constant is introduced by `axiomatization` and its defining equation is an axiom of the form `f x := body`.

$$\frac{a := b \quad Q\,b}{Q\,a}\ (\textsc{defE}) \qquad \frac{a := b \quad Q\,a}{Q\,b}\ (\textsc{defI})$$

```
  def :: ‹'a ⇒ 'a ⇒ o› (infix ‹:=› 10)
where
  defE: ‹⟦a := b; Q b⟧ ⟹ Q a› and
  defI: ‹⟦a := b; Q a⟧ ⟹ Q b›
```

`a := b` does not assert `a = b`. It asserts that `a` and `b` are interchangeable in every context `Q`, whether or not either denotes a value. That is what makes `omega := omega` harmless: `omega` is interchangeable with itself and `omega N` is underivable. The two rules are the unfolding and folding directions, and the `unfold_def` method of `unfold_def.ML` applies them.

2.9 Arithmetic

The standard functions are introduced by exactly this mechanism.

```
  add   :: "num ⇒ num ⇒ num"  (infixl "+" 60) and
  sub   :: "num ⇒ num ⇒ num"  (infixl "-" 60) and
  mult  :: "num ⇒ num ⇒ num"  (infixl "*" 70) and
  div   :: "num ⇒ num ⇒ num"                  and
  less  :: "num ⇒ num ⇒ num"  (infix "<" 50)  and
  leq   :: "num ⇒ num ⇒ num"  (infix "≤" 50) and
  omega :: "'a"
where
  add_def:   "add x y  := if y = 0 then x else S(add x (P y))"       and
  sub_def:   "sub x y  := if y = 0 then x else P(sub x (P y))"       and
  mult_def:  "mult x y := if y = 0 then 0 else (x + mult x (P y))"   and
  leq_def:   "leq x y  := if x = 0 then 1
                          else if y = 0 then 0
                          else (leq (P x) (P y))"                    and
  less_def:  "less x y := if y = 0 then 0
                          else if x = 0 then 1
                          else (less (P x) (P y))"                   and
  div_def:   "div x y  := if x < y = 1 then 0 else S(div (x - y) y)" and
  omega_def: "omega    := omega"
```

Comparison returns a term rather than a formula, so `x ≤ y` evaluates to `1` or `0` and the associated formula is `x ≤ y = 1`. The constant `omega` is included to exercise the claim that a divergent definition is admissible. Termination lemmas such as `add_terminates`, `leq_terminates` and `less_terminates` are proved by induction, as is termination of Ackermann's function, which is defined in the same file and is not primitive recursive.

2.10 Induction principles

Two derived induction principles carry most of the later work.

```
lemma strong_induction [consumes 1, case_names HQ Base Step]:
  shows "a N ⟹ Q 0 ⟹ (⋀x. x N ⟹ (⋀y. y N ⟹ y≤x = 1 ⟹ Q y) ⟹ (Q S(x))) ⟹ Q a"
```

```
lemma list_induct [case_names HQ Nil Cons]:
  "a N ⟹ Q Nil ⟹ (⋀x xs. x N ⟹ xs N ⟹ Q xs ⟹ Q (Cons x xs)) ⟹ Q a"
```

Strong induction is stated with the bounded hypothesis `y ≤ x = 1`. It is what makes recursion over the structure of an encoded term legitimate: a component of a code is numerically smaller

than the code, so structural recursion becomes bounded numerical recursion. `list_induct` is derived from it through the encoding of lists below.

2.11 Pairing

Pairing is the bridge from numbers to syntax. It is defined recursively rather than by a closed formula.

```
axiomatization cpair :: "num ⇒ num ⇒ num" where
  cpair_def: "cpair x y := if y = 0 then div (x * S(x)) 2
                           else cpair x P(y) + x + y + 1"
```

The syntax `⟨x, y⟩` abbreviates `cpair x y` and nests to the right for tuples. The inverses are recursive as well.

```
  cpx :: "num ⇒ num" and
  cpy :: "num ⇒ num"
where
  cpx_def: "cpx x := if x = 0 then 0
                     else if cpx (P x) = 0 then S(cpy P(x))
                     else P(cpx (P x))" and
  cpy_def: "cpy x := if cpx (P x) = 0 then 0
                     else S(cpy (P x))"
```

`cpi n x` projects the nth component of a right-nested tuple by iterating `cpy` and applying `cpx`.

```
  cpi' :: "num ⇒ num ⇒ num"
where
  cpi'_def: "cpi' n x := if n = 0 then 0
                         else if n = 1 then x
                         else cpy (cpi' (n-1) x)"
```

The properties established for the pairing are exactly the ones the encoding of §5 needs.

`cpair_terminates`	`x N ⟹ y N ⟹ ⟨x, y⟩ N`
`cpair_suc`	`x N ⟹ y N ⟹ ⟨x, S(y)⟩ = ⟨x, y⟩ + x + S(y) + 1`
`cpx_proj`, `cpy_proj`	the projection equations
`cpair_inj`	injectivity
`cpair_surjective`	every number is a pair
`cpair_reconstr`	`a N ⟹ ⟨cpx a, cpy a⟩ = a`
`cpx_mono`, `cpy_mono`	each projection is bounded by its argument

```
lemma cpx_proj [simp]: "a N ⟹ b N ⟹ cpx ⟨a, b⟩ = a"
apply (rule conjE1)
apply (rule cpx_cpy_proj)
apply (assumption)+
done

(* separated lemma for cpy*)
lemma cpy_proj [simp]: "a N ⟹ b N ⟹ cpy ⟨a, b⟩ = b"
```

```
lemma cpair_inj:
  assumes eq: "⟨a, b⟩ = ⟨c, d⟩"
  shows "a N ⟹ b N ⟹ c N ⟹ d N ⟹ a = c ∧ b = d"
```

```
lemma cpair_surjective [auto]:
  assumes a_nat : " a N"
  shows  "∃b c. a = ⟨b,c⟩"
```

```
lemma cpy_mono [simp]: "x N ⟹ cpy x ≤ x = 1"
apply (induct x)
proof (simp add: cpy_suc)+
  case (Step xa)
    show "x N ⟹ xa N ⟹ cpy xa ≤ xa = 1 ⟹
      (if cpx xa = 0 then 0 else S(cpy xa)) ≤ (S xa) = 1"
      by (cases bool: "cpx xa = 0", simp+)
qed


lemma cpx_mono [simp]: "x N ⟹ cpx x ≤ x = 1"
```

The Cantor pairing is a bijection between $\mathbb{N}^2$ and $\mathbb{N}$, and both directions of that fact are used later: injectivity gives the decomposition properties of the encoding, and surjectivity gives `cpair_reconstr`, which says that a number can always be taken apart and put back together. Monotonicity feeds strong induction: since `cpx x ≤ x` and `cpy x ≤ x`, a recursion that descends into components terminates.

2.12 Lists

Hypothesis sets, assignments, definition lists and proofs are all lists, so lists are the last piece of QGA infrastructure. `Nil` is 0 and `Cons` is a shifted pair, which makes the list encoding a bijection with $\mathbb{N}$ and removes the need for any well-formedness predicate on list codes.

```
type_synonym List = num

definition Nil :: "List" where
  "Nil ≡ 0"

definition Cons :: "num ⇒ List ⇒ List" where
  "Cons n xs ≡ ⟨n, xs⟩ + 1"

definition list_hd :: "List ⇒ num" where
  "list_hd x ≡ cpx (P x)"

definition list_tl :: "List ⇒ List" where
  "list_tl x ≡ cpy (P x)"
```

The derived facts are the expected ones: `nil_nat`, `cons_nat: ⟦n N; xs N⟧ ⟹ Cons n xs N`, disjointness of the two constructors, the projection equations `list_hd (Cons n xs) = n` and `list_tl (Cons n xs) = xs`, reconstruction `cons_reconstr: ⟦x N; ¬(x = Nil)⟧ ⟹ Cons (list_hd x) (list_tl x) = x`, injectivity `cons_inj`, exhaustiveness `list_cases: x N ⟹ (x = Nil) ∨ (∃n xs. x = Cons n xs)`, and `list_induct`. Together these are the characteristic properties of an inductive datatype, obtained with no datatype package.

Four list functions are defined recursively and used throughout §3.

```
axiomatization mem :: "num ⇒ List ⇒ o" (infixr "∈" 75) where
  mem_def: "mem x G := if G = Nil then False
                       else if list_hd G = x then True
                       else mem x (list_tl G)"
```

```
axiomatization nth :: "num ⇒ List ⇒ num" where
  nth_def: "nth i xs := if xs = Nil then 0
                        else if i = 0 then list_hd xs
                        else nth (i - 1) (list_tl xs)"
```

```
axiomatization len :: "List ⇒ num" where
  len_def: "len xs := if xs = Nil then 0 else S (len (list_tl xs))"
```

```
axiomatization subset :: "List ⇒ List ⇒ o" where
  subset_def: "subset G' G :=
    if G' = Nil then True
    else mem (list_hd G') G ∧ subset (list_tl G') G"
```

The two habeas quid results about them that the proof checker relies on are `mem_bool: ⟦x N; G N⟧ ⟹ mem x G B` and `nth_in_range_N: ⟦xs N; i < len xs = 1⟧ ⟹ nth i xs N`.

2.13 Summary

Table 1 collects the definitions of `GD.thy` in the notation of QGA rather than in Isabelle syntax. A definition written with $\triangleq$ is an abbreviation, unfolded by the framework. A definition written with $:=$ is a QGA recursive definition in the sense of §2.8, admitted with no termination obligation and used only through DEFE and DEFI.

3 BGA arithmetized in QGA

`BGA_on_GA.thy` encodes basic grounded arithmetic as QGA natural numbers and proves its proof checker sound with respect to the encoded semantics. The file is a tower of locales, so the whole argument is carried out under a fixed, small set of assumptions about the encoding and no concrete Gödel numbering appears in it.

This section walks through the locales in order and explains every constant they fix. Throughout, $\Gamma \vdash \varphi$ is a BGA judgment, π and ρ range over encoded proofs, A over assignments, and D over the background definition list. Notation like $q[v_i \mapsto a]$ abbreviates `subst_F q i a`.

3.1 The system being encoded

BGA has terms and formulas, and no logical connectives at all.

$$\begin{array}{lcl} t & ::= & v_j \mid 0 \mid \mathsf{S}(t) \mid \mathsf{P}(t) \mid t\,0?\,t : t \mid d_i(t,t) \\ \varphi & ::= & t = t \mid t \neq t \end{array}$$

A variable v_j is a natural number j, and its value under an assignment is the jth entry of the assignment list. BGA has no binders, so nothing scopes over a variable and the index is simply a position. Conditional evaluation is if-zero-else, which keeps terms and formulas from being mutually recursive. Function application is restricted to two arguments, and the functions come from a fixed background list D, where position i holds the body of $d_i(v_0, v_1)$. Disequality is a primitive, so $a \neq b$ is not the negation of $a = b$; BGA has no negation to form. A judgment is a sequent $\Gamma \vdash \varphi$ with Γ a finite list of formulas.

3.2 Codes

There is one QGA type, `num`. The type synonyms below are documentation.

Abbreviations	
$a \neq b$	$\triangleq \neg(a = b)$
$p\,\mathsf{B}$	$\triangleq p \vee \neg p$
$a\,\mathsf{N}$	$\triangleq a = a$
$p \wedge q$	$\triangleq \neg(\neg p \vee \neg q)$
$p \longrightarrow q$	$\triangleq \neg p \vee q$
$p \longleftrightarrow q$	$\triangleq (p \longrightarrow q) \wedge (q \longrightarrow p)$
True	$\triangleq 0 = 0$
False	$\triangleq \mathsf{S}(0) = 0$
$x > y$	$\triangleq 1 - (x \leq y)$
$x \geq y$	$\triangleq 1 - (x < y)$
Nil	$\triangleq 0$
$\mathsf{Cons}\ n\ xs$	$\triangleq \langle n, xs\rangle + 1$
$\mathsf{hd}\ x$	$\triangleq \mathsf{cpx}\,\mathsf{P}(x)$
$\mathsf{tl}\ x$	$\triangleq \mathsf{cpy}\,\mathsf{P}(x)$
Recursive definitions	
$x + y$	$:= \mathsf{if}\ y = 0\ \mathsf{then}\ x\ \mathsf{else}\ \mathsf{S}(x + \mathsf{P}(y))$
$x - y$	$:= \mathsf{if}\ y = 0\ \mathsf{then}\ x\ \mathsf{else}\ \mathsf{P}(x - \mathsf{P}(y))$
$x * y$	$:= \mathsf{if}\ y = 0\ \mathsf{then}\ 0\ \mathsf{else}\ x + (x * \mathsf{P}(y))$
$x \leq y$	$:= \mathsf{if}\ x = 0\ \mathsf{then}\ 1\ \mathsf{else if}\ y = 0\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{P}(x) \leq \mathsf{P}(y)$
$x < y$	$:= \mathsf{if}\ y = 0\ \mathsf{then}\ 0\ \mathsf{else if}\ x = 0\ \mathsf{then}\ 1\ \mathsf{else}\ \mathsf{P}(x) < \mathsf{P}(y)$
$\mathsf{div}\ x\ y$	$:= \mathsf{if}\ (x < y) = 1\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{S}(\mathsf{div}\ (x - y)\ y)$
ω	$:= \omega$
$\langle x, y\rangle$	$:= \mathsf{if}\ y = 0\ \mathsf{then}\ \mathsf{div}\ (x * \mathsf{S}(x))\ 2\ \mathsf{else}\ \langle x, \mathsf{P}(y)\rangle + x + y + 1$
$\mathsf{cpx}\ x$	$:= \mathsf{if}\ x = 0\ \mathsf{then}\ 0\ \mathsf{else if}\ \mathsf{cpx}\,\mathsf{P}(x) = 0\ \mathsf{then}\ \mathsf{S}(\mathsf{cpy}\,\mathsf{P}(x))\ \mathsf{else}\ \mathsf{P}(\mathsf{cpx}\,\mathsf{P}(x))$
$\mathsf{cpy}\ x$	$:= \mathsf{if}\ \mathsf{cpx}\,\mathsf{P}(x) = 0\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{S}(\mathsf{cpy}\,\mathsf{P}(x))$
$\mathsf{cpi}'\ n\ x$	$:= \mathsf{if}\ n = 0\ \mathsf{then}\ 0\ \mathsf{else if}\ n = 1\ \mathsf{then}\ x\ \mathsf{else}\ \mathsf{cpy}\ (\mathsf{cpi}'\ (n - 1)\ x)$
$x \in G$	$:= \mathsf{if}\ G = \mathsf{Nil}\ \mathsf{then}\ \mathsf{False}\ \mathsf{else if}\ \mathsf{hd}\,G = x\ \mathsf{then}\ \mathsf{True}\ \mathsf{else}\ x \in \mathsf{tl}\,G$
$\mathsf{nth}\ i\ xs$	$:= \mathsf{if}\ xs = \mathsf{Nil}\ \mathsf{then}\ 0\ \mathsf{else if}\ i = 0\ \mathsf{then}\ \mathsf{hd}\,xs\ \mathsf{else}\ \mathsf{nth}\ (i - 1)\ (\mathsf{tl}\,xs)$
$\mathsf{len}\ xs$	$:= \mathsf{if}\ xs = \mathsf{Nil}\ \mathsf{then}\ 0\ \mathsf{else}\ \mathsf{S}(\mathsf{len}\ (\mathsf{tl}\,xs))$
$\mathsf{subset}\ G'\ G$	$:= \mathsf{if}\ G' = \mathsf{Nil}\ \mathsf{then}\ \mathsf{True}\ \mathsf{else}\ \mathsf{hd}\,G' \in G \wedge \mathsf{subset}\ (\mathsf{tl}\,G')\ G$
$\mathsf{ack}\ x\ y$	$:= \mathsf{if}\ x = 0\ \mathsf{then}\ y + 1\ \mathsf{else if}\ y = 0\ \mathsf{then}\ \mathsf{ack}\,\mathsf{P}(x)\,1\ \mathsf{else}\ \mathsf{ack}\,\mathsf{P}(x)\ (\mathsf{ack}\ x\ \mathsf{P}(y))$

Table 1: Definitions of QGA. $\langle x, y\rangle$ is the Cantor pairing and cpx, cpy its projections.

```
type_synonym tm  = num  (* BGA Term *)
type_synonym fm  = num  (* BGA Formula *)
type_synonym asn = num  (* Assignment *)
type_synonym dfn = num  (* Definition *)
type_synonym pf = num  (* Proof: List of judgements *)
type_synonym val = num  (* Evaluated Value *)

type_synonym hyp = num  (* Hypothesis: list/set of formulas *)
type_synonym jdg = num  (* Judgment: (hyp, fm) *)

abbreviation hyp_of :: "jdg ⇒ hyp" where "hyp_of J ≡ cpx J"
abbreviation conc_of :: "jdg ⇒ fm" where "conc_of J ≡ cpy J"

abbreviation mk_jdg :: "hyp ⇒ fm ⇒ jdg" (infix "⊩⊢" 50)
  where "G ⊩⊢ c ≡ ⟨G, c⟩"
abbreviation emptyH :: "hyp" ("∅")
  where "∅ ≡ Nil"
abbreviation cons  :: "fm ⇒ List ⇒ List" (infixr "▷" 65)
  where "f ▷ G ≡ Cons f G"
```

A judgment is the pair `⟨G, c⟩`, written `G ⊩⊢ c`, with `hyp_of` and `conc_of` as projections. A hypothesis set is a list of formulas and a proof is a list of judgments, read head first, each justified by the judgments after it.

3.3 The locale tower

Locale	Extends	Fixes
`suff_syntax`	—	`mk_eq`, `mk_neq`, `dfns`, `is_valid_proof`
`suff_semantics`	`suff_syntax`	`evals`, `sat_fm`, `sat_hyp`
`consistent`	`suff_semantics`	—
`bga_encoding`	—	the six encoder operations, the three freshness predicates
`bga_dfns`	`bga_encoding`	`dfns`, `dfn_is`
`bga_fuel_semantics`	`bga_dfns`	`eval_fuel`
`bga_subst`	`bga_encoding`	`subst_T`, `subst_F`, `subst_body`
`bga_fuel_subst_semantics`	`bga_fuel_semantics`, `bga_subst`	`asn_put`
`bga_subst_rule`	`bga_subst`	`check_template`, `rep_vars_T`, `rep_vars_F`, `find_phi`, `find_eq`, `check_subst`
`bga_eq_rule`	`bga_encoding`	`check_eq_rules`, `check_neq_rules`
`bga_ind_rule`	`bga_subst_rule`	`check_ind_template`, `find_ind_base`, `check_ind`
`bga_struct_rule`	`bga_encoding`	`find_cut`, `check_cut`, `find_struct`, `check_struct`
`bga_app_rule`	`bga_subst_rule`, `bga_dfns`	`app_try`, `app_y`, `app_x`, `app_d`, `check_app`
`bga_proof_check`	the five rule locales	`valid_step`, `check_list`, `is_valid_proof`
`bga_fuller`	`bga_fuel_subst_semantics`, `bga_proof_check`	—

Two locales are defined in the file and not used by `bga_fuller`: `bga_semantics`, which fixes the exact evaluator `eval`, and `bga_subst_semantics`, which combines it with substitution. They are described in §3.8 because the exact evaluator is the clearest statement of what the encoded semantics means.

Almost every constant is fixed with an assumption of the form `f x := body` rather than being defined. A locale cannot make a recursive definition, and in QGA it does not need to: fixing a constant together with its `:=` equation is exactly what §2.8 admits, and the equation is all the proofs ever use.

3.4 The consistency skeleton

Before any encoding is fixed, the shape of the final argument is stated.

```
locale suff_syntax =
  (*Encoding  *)
  fixes mk_eq :: "tm ⇒ tm ⇒ fm"
  fixes mk_neq :: "tm ⇒ tm ⇒ fm"
  (* Fixed Definition List*)
  fixes dfns :: "dfn"
  (*Provability. is_valid_proof \<lbrace> p \<rbrace> \<lbrace>Γ ⊢ f \<rbrace> ≡ p \<P> ⟦ Γ ⊢
    ↪ f ⟧*)
  fixes is_valid_proof :: "pf ⇒ jdg ⇒ o"

  (*Habeas Quid for Syntax Constructors *)
  assumes mk_eq_N:  "⟦a N; b N⟧ ⟹ mk_eq a b N"
  assumes mk_neq_N: "⟦a N; b N⟧ ⟹ mk_neq a b N"

  (*  Habeas Quid for the Proof Checker *)
  assumes proof_bool: "⟦p N; J N⟧ ⟹ ((is_valid_proof p J) B)"
```

`mk_eq :: tm ⇒ tm ⇒ fm`	builds the code of $a = b$ from the codes of a and b
`mk_neq :: tm ⇒ tm ⇒ fm`	the same for $a \neq b$
`dfns :: dfn`	the code of the background definition list D
`is_valid_proof :: pf ⇒ jdg ⇒ o`	"π is a proof of J"

The assumptions are only habeas quid: encoded formulas are numbers, and `is_valid_proof p J` is a grounded formula whenever `p` and `J` are numbers. The second is the grounded-logic form of decidability of proof checking, and it is what makes the concluding proof by contradiction available.

```
locale suff_semantics = suff_syntax +
  (*Semantics. sat checks whether the encoding of a formula is satisfied by an assignment.
    ↪ eval reduces a term under an assignment*)

  fixes evals :: "tm ⇒ asn ⇒ val ⇒ o"
  fixes sat_fm :: "fm ⇒ asn ⇒ o"
  fixes sat_hyp :: "hyp ⇒ asn ⇒ o"

  assumes sat_hyp_nil: "sat_hyp Nil A"

  (* Evaluation is deterministic *)
  assumes evals_det: "⟦t N; A N; evals t A r; evals t A q⟧ ⟹ r = q"

  (* What equations mean in the model: the two sides evaluate to a common value
     (resp. to distinct values). *)
  assumes sat_eqE:
    "⟦a N; b N; sat_fm (mk_eq a b) A;
      ⋀q. ⟦q N; evals a A q; evals b A q⟧ ⟹ R⟧ ⟹ R"
```

```
  assumes sat_neqE:
    "⟦a N; b N; sat_fm (mk_neq a b) A;
      ⋀x y. ⟦x N; y N; evals a A x; evals b A y; x ≠ y⟧ ⟹ R⟧ ⟹ R"
```

`evals :: tm ⇒ asn ⇒ val ⇒ o`	"t evaluates to r under A"
`sat_fm :: fm ⇒ asn ⇒ o`	"the formula coded by f holds under A"
`sat_hyp :: hyp ⇒ asn ⇒ o`	"every formula in Γ holds under A"

Three assumptions: the empty hypothesis list is satisfied by every assignment, evaluation is deterministic, and a satisfied encoded equation yields a common value for its two sides while a satisfied encoded disequality yields two distinct values.

```
locale consistent =  suff_semantics +
  (* Valid proofs yield satisfied formulas *)
  assumes soundness: "⟦is_valid_proof p J; sat_hyp (hyp_of J) A; A N⟧ ⟹ sat_fm (conc_of J) A"
begin

lemma syntactically_consistent:
  assumes a_nat: "a N"
  assumes b_nat: "b N"
  assumes p1_nat: "p1 N"
  assumes p2_nat: "p2 N"
  shows " ¬ (is_valid_proof p1 ⟨Nil, (mk_eq a b)⟩ ∧ is_valid_proof p2 ⟨Nil, (mk_neq a b)⟩)"
```

`consistent` adds soundness of the checker and derives syntactic consistency. The derivation is short. Assume both proofs exist. The statement is grounded by `proof_bool`, so `contradiction` applies. Soundness under the empty hypothesis list and the assignment `0` gives `sat_fm (mk_eq a b) 0` and `sat_fm (mk_neq a b) 0`. The first produces a value `q` reached by both `a` and `b`, the second produces `x` and `y` with `x ≠ y`, determinism forces `x = q = y`, and `exF` closes the goal. The habeas quid premises `a N`, `b N`, `p1 N` and `p2 N` are needed before the formula being contradicted can be shown grounded.

3.5 Encoded syntax

Terms and formulas are tagged pairs, with six term tags and two formula tags.

```
  abbreviation T_VAR  :: num where "T_VAR ≡ 0"
  abbreviation T_ZERO :: num where "T_ZERO ≡ 1"
  abbreviation T_SUC  :: num where "T_SUC ≡ 2"
  abbreviation T_PRED :: num where "T_PRED ≡ 3"
  abbreviation T_IFZ  :: num where "T_IFZ ≡ 4"
  abbreviation T_APP  :: num where "T_APP ≡ 5"

  (* Formula Constructor Tags *)
  abbreviation F_EQ   :: num where "F_EQ ≡ 0"
  abbreviation F_NEQ  :: num where "F_NEQ ≡ 1"
```

Tag	Constructor	Notation	Payload
`T_VAR`	variable	v_j	j
`T_ZERO`	zero	0	ignored
`T_SUC`	successor	$\mathsf{S}(a)$	a
`T_PRED`	predecessor	$\mathsf{P}(a)$	a
`T_IFZ`	if-zero-else	$c\ 0?\ a : b$	$\langle c, \langle a, b\rangle\rangle$
`T_APP`	application	$d_i(a, b)$	$\langle i, \langle a, b\rangle\rangle$
`F_EQ`	equality	$a = b$	$\langle a, b\rangle$
`F_NEQ`	disequality	$a \neq b$	$\langle a, b\rangle$

3.6 `bga_encoding`

```
locale bga_encoding =
  (*Term Encoding *)
  (* Tag of the current Constructor *)
  fixes tag_T :: "tm ⇒ tmtag"
  (* Encoding of the argument *)
  fixes load_T :: "tm ⇒ tm"
  (* Encodes the term *)
  fixes pack_T :: "tmtag ⇒ tm ⇒ tm"

  (* Formula Encoding *)
  fixes tag_F :: "fm ⇒ fmtag"
  fixes load_F :: "fm ⇒ tm"
  fixes pack_F :: "fmtag ⇒ fm ⇒ fm"

(* Fixing Definition*)
  fixes fresh_T :: "num ⇒ tm ⇒ o"
  fixes fresh_F :: "num ⇒ fm ⇒ o"
  fixes fresh_H :: "num ⇒ hyp ⇒ o"

  assumes fresh_T_def:
    "fresh_T k t :=
       if tag_T t = T_VAR then load_T t < k = 1
       else if tag_T t = T_ZERO then True
       else if tag_T t = T_SUC then fresh_T k (load_T t)
       else if tag_T t = T_PRED then fresh_T k (load_T t)
       else if tag_T t = T_IFZ then
         fresh_T k (cpx (load_T t)) ∧
         fresh_T k (cpx (cpy (load_T t))) ∧
         fresh_T k (cpy (cpy (load_T t)))
       else if tag_T t = T_APP then
         fresh_T k (cpx (cpy (load_T t))) ∧
         fresh_T k (cpy (cpy (load_T t)))
       else False"
  assumes fresh_F_def: "fresh_F k f := fresh_T k (cpx (load_F f)) ∧ fresh_T k (cpy (load_F f))"
  assumes fresh_H_def: "fresh_H k G := if G = Nil then True else fresh_F k (list_hd G) ∧
    ↪  fresh_H k (list_tl G)"
```

`tag_T :: tm ⇒ tmtag`	the constructor tag of a term code
`load_T :: tm ⇒ tm`	the payload of a term code
`pack_T :: tmtag ⇒ tm ⇒ tm`	builds a term code from tag and payload
`tag_F`, `load_F`, `pack_F`	the same three for formula codes
`fresh_T :: num ⇒ tm ⇒ o`	every variable in the term has index below k
`fresh_F :: num ⇒ fm ⇒ o`	the same for a formula
`fresh_H :: num ⇒ hyp ⇒ o`	the same for every formula in a list

Freshness is defined by recursion over the tag, and in mathematical notation it reads

$$\mathsf{fresh}_T(k,t) = \begin{cases} j < k & t = v_j \\ \top & t = 0 \\ \mathsf{fresh}_T(k,a) & t = \mathsf{S}(a) \text{ or } t = \mathsf{P}(a) \\ \mathsf{fresh}_T(k,c) \wedge \mathsf{fresh}_T(k,a) \wedge \mathsf{fresh}_T(k,b) & t = (c\,0?\,a : b) \\ \mathsf{fresh}_T(k,a) \wedge \mathsf{fresh}_T(k,b) & t = d_i(a,b) \\ \bot & \text{tag out of range} \end{cases}$$

with $\mathsf{fresh}_F(k,\varphi)$ the conjunction over the two sides of φ and $\mathsf{fresh}_H(k,\Gamma)$ the conjunction over the members of Γ. The last line is the treatment of a number whose tag is not one of the six: it is fresh for no k. Freshness is used once, in the induction rule of §3.14, to guarantee that the induction variable does not occur in the hypotheses.

The assumptions on the interface are the following, and they are the only properties of the encoding used anywhere in the development.

```
assumes pack_F_N: "⟦t N; L N⟧ ⟹ pack_F t L N"
assumes tag_F_N:  "f N ⟹ tag_F f N"
assumes load_F_N: "f N ⟹ load_F f N"

assumes pack_T_N: "⟦t N; L N⟧ ⟹ (pack_T t L) N"
assumes tag_T_N:  "t N ⟹ tag_T t N"
assumes load_T_N: "t N ⟹ load_T t N"

(*Injective *)
assumes tag_pack_F:  "⟦t N; L N⟧ ⟹ tag_F (pack_F t L) = t"
assumes load_pack_F: "⟦t N; L N⟧ ⟹ load_F (pack_F t L) = L"

assumes tag_pack_T:  "⟦t N; L N⟧ ⟹ tag_T (pack_T t L) = t"
assumes load_pack_T: "⟦t N; L N⟧ ⟹ load_T (pack_T t L) = L"

(*Surjective *)
assumes pack_tag_F:  "f N ⟹ pack_F (tag_F f) (load_F f) = f"
assumes pack_tag_T:  "t N ⟹ pack_T (tag_T t) (load_T t) = t"

(*Structural Decrease (For Induction) *)
(* This will require load of atomic terms(without arguments) to return 0 *)
assumes decrease_F: "⟦f N; f ≠ 0 ⟧ ⟹ load_F f < f = 1"
assumes decrease_T: "⟦t N; t ≠ 0⟧ ⟹ load_T t < t = 1"

assumes tag_T_zero: "tag_T 0 = T_ZERO"

(*Monotone assumptions for Bounding *)
assumes mono_pack_T: "⟦tg N; x N; y N; x ≤ y = 1⟧ ⟹ (pack_T tg x ≤ pack_T tg y = 1)"
assumes mono_pack_F: "⟦tg N; x N; y N; x ≤ y = 1⟧ ⟹ (pack_F tg x ≤ pack_F tg y = 1)"
```

habeas quid	`pack`, `tag` and `load` preserve `N`
injectivity	`tag (pack t L) = t` and `load (pack t L) = L`
retraction	`pack (tag f) (load f) = f`
decrease	`f ≠ 0 ⟹ load f < f = 1`, payloads are strictly smaller
base case	`tag_T 0 = T_ZERO`, the code 0 is the term 0
monotonicity	`pack` is monotone in its payload

Remark 3.1 (Injective, not surjective)**.** What the concrete encoding of §5 provides, and all that is used, is an *injective* map from BGA syntax into $\mathbb{N}$. It is not surjective: a number whose tag lies outside the constructor range encodes no BGA term, and no attempt is made to exclude

such numbers or to recognize the well-formed ones. The assumption that does hold for every number is the retraction property `pack (tag f) (load f) = f`, which follows from the Cantor pairing being a bijection on $\mathbb{N}$ and says only that a code can be taken apart and put back together. Ill-formed codes are harmless: the evaluator gives them the application semantics, freshness rejects them, and soundness never needs them to denote anything.

The decrease and monotonicity assumptions are what allow `strong_induction` to serve as structural induction over encoded terms. Decrease gives the descent: the payload of a nonzero code is a strictly smaller number, so recursion into subterms is bounded numerical recursion. Monotonicity is used differently, to justify the bounds of the searches in §3.12. The first lemma proved from them is `fresh_T_bool`, that freshness is a grounded formula.

3.7 `bga_dfns`

```
locale bga_dfns = bga_encoding +
  fixes dfns :: "dfn"
  fixes dfn_is :: "dfn ⇒ num ⇒ tm ⇒ o"
  assumes dfns_N: "dfns N"
  (* lazy range check instea of direct conjunction which makes it kind of awkward to work with
   ↪  *)
  assumes dfn_is_def: "dfn_is d k b := if d < len dfns = 1 then nth d dfns = b ∧ fresh_T k b
   ↪  else False"
```

`dfns :: dfn`	the code of the definition list D, a list of term codes
`dfn_is :: dfn ⇒ num ⇒ tm ⇒ o`	"d is in range, D_d is b, and b has arity $\leq k$"

Written out, dfn_is(d, k, b) holds when $d < |D|$, $D_d = b$ and fresh$_T(k, b)$. The range test is written as a nested conditional rather than a conjunction so that nth $d\, D$ is never mentioned when d is out of range. The application rule instantiates it at $k = 2$, which is the sense in which BGA definitions are binary: the body may mention v_0 and v_1 and nothing else.

3.8 Semantics

Two evaluators appear in the file. The direct one, fixed by `bga_semantics`, is the natural reading of the BGA big-step semantics, and writing it down at all depends on QGA tolerating divergent definitions.

```
  assumes eval_def: "eval t A :=
    if tag_T t = T_VAR then nth (load_T t) A
    else if tag_T t = T_ZERO then 0
    else if tag_T t = T_SUC then S(eval (load_T t) A)
    else if tag_T t = T_PRED then P(eval (load_T t) A)
    else if tag_T t = T_IFZ then
      (if eval (cpx (load_T t)) A = 0
         then eval (cpx (cpy (load_T t))) A
         else eval (cpy (cpy (load_T t))) A)
    else
      (if eval (cpx (cpy (load_T t))) A = eval (cpx (cpy (load_T t))) A
       then (if eval (cpy (cpy (load_T t))) A = eval (cpy (cpy (load_T t))) A
             then eval (nth (cpx (load_T t)) dfns)
                    ((eval (cpx (cpy (load_T t))) A)▹ ((eval (cpy (cpy (load_T t))) A) ▹ Nil))
             else 0)
       else 0)"
```

In mathematical notation, with A an assignment,

$$\mathsf{eval}(t, A) = \begin{cases} \mathsf{nth}\, j\, A & t = v_j \\ 0 & t = 0 \\ \mathsf{S}(\mathsf{eval}(a, A)) & t = \mathsf{S}(a) \\ \mathsf{P}(\mathsf{eval}(a, A)) & t = \mathsf{P}(a) \\ \mathsf{eval}(a, A) \text{ or } \mathsf{eval}(b, A) & t = (c\, 0?\, a : b), \text{ on } \mathsf{eval}(c, A) = 0 \\ \mathsf{eval}(D_i,\ [\mathsf{eval}(a, A),\ \mathsf{eval}(b, A)]) & t = d_i(a, b) \end{cases}$$

The application case in the sources is wrapped in two guards of the form `eval a A = eval a A`. Those are habeas quid tests: the arguments are evaluated first, and the body is entered only if both terminate, which makes the encoded semantics call by value.

The consistency argument runs through the step-indexed variant instead, fixed by `bga_fuel_semantics`. Its result is offset by one, so that $\mathsf{eval_fuel}(k, t, A) = 0$ means "no result within budget k" and $\mathsf{eval_fuel}(k, t, A) = 1 + v$ means "the value is v". The offset is what lets a single `num` carry both outcomes, at the cost of an explicit $\mathsf{P}(\cdot)$ on every recursive result.

```
locale bga_fuel_semantics = bga_dfns +
  fixes eval_fuel :: "num ⇒ tm ⇒ asn ⇒ val"
  assumes eval_fuel_def: "eval_fuel k t A :=
    if k = 0 then 0
    else if tag_T t = T_VAR then S (nth (load_T t) A)
    else if tag_T t = T_ZERO then S 0
    else if tag_T t = T_SUC then
      if eval_fuel (P k) (load_T t) A = 0 then 0
      else S (S (P (eval_fuel (P k) (load_T t) A)))
    else if tag_T t = T_PRED then
      if eval_fuel (P k) (load_T t) A = 0 then 0
      else S (P (P (eval_fuel (P k) (load_T t) A)))
    else if tag_T t = T_IFZ then
      if eval_fuel (P k) (hyp_of (load_T t)) A = 0 then 0
      else if P (eval_fuel (P k) (hyp_of (load_T t)) A) = 0
      then eval_fuel (P k) (hyp_of (conc_of (load_T t))) A
      else eval_fuel (P k) (conc_of (conc_of (load_T t))) A
    else
      if eval_fuel (P k) (hyp_of (conc_of (load_T t))) A = 0 then 0
      else if eval_fuel (P k) (conc_of (conc_of (load_T t))) A = 0 then 0
      else eval_fuel (P k) (nth (hyp_of (load_T t)) dfns)
        (P (eval_fuel (P k) (hyp_of (conc_of (load_T t))) A) ▹
         P (eval_fuel (P k) (conc_of (conc_of (load_T t))) A) ▹ Nil)"
```

```
definition evals :: "tm ⇒ asn ⇒ val ⇒ o" where
  "evals t A r ≡ ∃k. eval_fuel k t A = S r"
```

`evals t A r` says that t evaluates to r under A for some budget:

$$\mathsf{evals}(t, A, r) \quad \equiv \quad \exists k.\ \mathsf{eval_fuel}(k, t, A) = \mathsf{S}(r).$$

Satisfaction comes in the same two forms. For the direct evaluator, `bga_semantics` defines

```
definition sat :: "num ⇒ num  ⇒ o" where
    "sat f A ≡ if tag_F f = 0
               then eval (cpx (load_F f)) A = eval (cpy (load_F f)) A
               else eval (cpx (load_F f)) A ≠ eval (cpy (load_F f)) A"

definition sat_hyp :: "hyp ⇒ asn ⇒ o" where
```

```
  "sat_hyp G A ≡ ∀f. (mem f G) ⟶ (sat f A)"
```

that is, $\mathsf{sat}(\varphi, A)$ compares eval of the two sides, with the comparison chosen by the formula tag, and sat_hyp quantifies over the members of the list. For the step-indexed evaluator, satisfaction of an equation also has to require both sides to converge:

```
definition sat_fuel :: "fm ⇒ asn ⇒ o" where
  "sat_fuel f A ≡ ∃x. ∃y. evals (hyp_of (load_F f)) A x ∧ (evals (conc_of (load_F f)) A y ∧
   ↪ (if tag_F f = F_EQ then x = y else x ≠ y))"
```

```
definition sat_hyp_fuel :: "hyp ⇒ asn ⇒ o" where
  "sat_hyp_fuel G A ≡ ∀f. f ∈ G ⟶ sat_fuel f A"
```

$$\mathsf{sat_fuel}(\varphi, A) \quad \equiv \quad \exists x, y.\ \mathsf{evals}(a, A, x) \wedge \mathsf{evals}(b, A, y) \wedge \begin{cases} x = y & \text{tag } \varphi = \mathsf{F_EQ} \\ x \neq y & \text{otherwise} \end{cases}$$

where a and b are the two sides of φ. A formula whose sides diverge is unsatisfied under every assignment, which is the property the consistency argument ultimately rests on.

3.9 Satisfaction at a fixed budget

`sat_fuel` is not known to be grounded. Unfolding it gives an existential over the budget, so deciding it would decide whether an evaluation converges. That matters for §4.4, where satisfaction of the hypotheses appears as the antecedent of an implication inside an induction motive, and ⟶ introduction requires the antecedent to be grounded.

The budget is the only source of the difficulty. `eval_fuel` is total, so at a fixed k every question about it is decided. A second layer of definitions pins the budget.

`ev_ok k t A`	$\triangleq \neg(\mathsf{eval_fuel}(k, t, A) = 0)$, success within k
`sat_at k f A`	both sides succeed within k and compare as the tag requires
`sat_hyp_at k G A`	$\triangleq \forall f.\, f \in G \longrightarrow \mathsf{sat_at}(k, f, A)$

`sat_at` is quantifier free, so `sat_at_bool` follows from `eval_fuel_N` and `eqBool`, on the pattern of `eval_fuel_zero_bool`. `sat_hyp_at_bool` goes by list induction, and its negative case has to refute a universal, since some member of the list may fail. That is the one place in the development where ¬∀I and ¬∀E of §2.6 are used.

Both predicates are monotone in the budget by `eval_fuel_success_add`, which gives the two bridges back to §3.8.

`sat_hyp_atD`	`sat_hyp_at k G A ⟹ sat_hyp_fuel G A`
`sat_hyp_at_exI`	`sat_hyp_fuel G A ⟹ ∃k. sat_hyp_at k G A`

The downward bridge is immediate. The upward one needs one budget that serves every member of the list, and `eval_fuel_success_add` supplies it by addition, the same device `eval_fuel_success_unique` uses to compare two runs.

The upward bridge has a subtlety. Proving it by list induction on G would put `sat_hyp_fuel G A` in the antecedent of the motive, which is the problem it exists to solve. It is therefore proved by induction over a sublist G' of a fixed G. The antecedent of the motive becomes `subset G' G`, which `subset_bool` decides, while `sat_hyp_fuel G A` stays outside the induction as an ordinary assumption. Instantiating at $G' = G$ through `subset_refl` gives the statement.

3.10 `bga_subst` and `asn_put`

```
locale bga_subst = bga_encoding +
  fixes subst_T        :: "tm ⇒ tm ⇒ tm ⇒ tm"
  (* subst_T t j v: Inside term 't', replace variable index 'j' with term 'v' *)
  assumes subst_T_def: "subst_T t j v :=
    if tag_T t = T_VAR then
      (if load_T t = j then v else t)
    else if tag_T t = T_ZERO then (pack_T T_ZERO 0)
    else if tag_T t = T_SUC then (pack_T T_SUC (subst_T (load_T t) j v))
    else if tag_T t = T_PRED then (pack_T T_PRED (subst_T (load_T t) j v))
    else if tag_T t = T_IFZ then
      (pack_T T_IFZ ⟨subst_T (cpx (load_T t)) j v
            , ⟨(subst_T (cpx (cpy (load_T t))) j v),
            (subst_T (cpy (cpy (load_T t))) j v)⟩⟩)
    else
      (pack_T T_APP ⟨(cpx (load_T t)), ⟨
             (subst_T (cpx (cpy (load_T t))) j v),
             (subst_T (cpy (cpy (load_T t))) j v)⟩⟩)"

    fixes subst_F        :: "fm ⇒ tm ⇒ tm ⇒ fm"
  (* subst_F f j v: Inside formula 'f', replace variable index 'j' with term 'v' *)
  assumes subst_F_def: "subst_F f j v :=
    if tag_F f = F_EQ then
      pack_F F_EQ ⟨(subst_T (cpx (load_F f)) j v), (subst_T (cpy (load_F f)) j v)⟩
    else
      pack_F F_NEQ ⟨(subst_T (cpx (load_F f)) j v), (subst_T (cpy (load_F f)) j v)⟩"

  fixes subst_body :: "tm ⇒ tm ⇒ tm ⇒ tm"
  (* subst_body b x y  =  b[0↦x, 1↦y] simultaneously *)
  assumes subst_body_def: "subst_body b x y :=
    if tag_T b = T_VAR then
      (if load_T b = 0 then x else if load_T b = 1 then y else b)
    else if tag_T b = T_ZERO then b
    else if tag_T b = T_SUC  then pack_T T_SUC  (subst_body (load_T b) x y)
    else if tag_T b = T_PRED then pack_T T_PRED (subst_body (load_T b) x y)
    else if tag_T b = T_IFZ  then
      pack_T T_IFZ ⟨subst_body (cpx (load_T b)) x y,
               ⟨subst_body (cpx (cpy (load_T b))) x y,
                subst_body (cpy (cpy (load_T b))) x y⟩⟩
```

```
  fixes asn_put :: "asn ⇒ num ⇒ val ⇒ asn"
  assumes asn_put_def: "asn_put A i v :=
    if i = 0 then
      if A = Nil then v ▹ Nil else v ▹ list_tl A
    else if A = Nil then 0 ▹ asn_put Nil (i - 1) v
    else list_hd A ▹ asn_put (list_tl A) (i - 1) v"
```

`subst_T :: tm ⇒ tm ⇒ tm ⇒ tm`	$t[v_j \mapsto v]$, replace variable j inside a term
`subst_F :: fm ⇒ tm ⇒ tm ⇒ fm`	the same inside a formula, side by side
`subst_body :: tm ⇒ tm ⇒ tm ⇒ tm`	$b[v_0 \mapsto x, v_1 \mapsto y]$, simultaneous
`asn_put :: asn ⇒ num ⇒ val ⇒ asn`	$A[i \mapsto v]$, update an assignment

All three substitutions recurse over the tag and rebuild with `pack_T`. Two details are worth noting. In the `T_ZERO` case `subst_T` returns `pack_T T_ZERO 0` rather than the original code, which normalizes the ignored payload of a zero. In the `T_APP` case the definition index `cpx (load_T t)` is carried through untouched, since it is not a term. `subst_body` is the two-variable version used by the application rule, and it exists separately because that rule substitutes both arguments at once. `asn_put A i v` walks down i entries and replaces the one it finds, padding with zeros when i runs past the end of the list, so that an assignment is a total map from indices to values.

3.11 The proof rules of BGA

The rules are given below in sequent form, and each is realized as one branch of the encoded checker. Here Γ is a list of encoded formulas, φ an encoded formula, and $\varphi[a]$ a formula with occurrences of a marked, which is realized by the template search of §3.12. The turnstile is the BGA sequent arrow, written $\Vdash$ in the sources.

$$\frac{}{\Gamma, \varphi \vdash \varphi}\ (\textsc{hyp}) \qquad \frac{}{\Gamma \vdash 0 = 0}\ (0\text{I})$$

$$\frac{\Gamma \vdash a = b}{\Gamma \vdash b = a}\ (=\text{S}) \qquad \frac{\Gamma \vdash a = b \quad \Gamma \vdash \varphi[a]}{\Gamma \vdash \varphi[b]}\ (=\text{E})$$

$$\frac{\Gamma \vdash a = b}{\Gamma \vdash \mathsf{S}(a) = \mathsf{S}(b)}\ (\text{S=I}) \qquad \frac{\Gamma \vdash \mathsf{S}(a) = \mathsf{S}(b)}{\Gamma \vdash a = b}\ (\text{S=E})$$

$$\frac{\Gamma \vdash a\,\mathsf{N}}{\Gamma \vdash \mathsf{P}(\mathsf{S}(a)) = a}\ (\text{PI}) \qquad \frac{\Gamma \vdash a\,\mathsf{N}}{\Gamma \vdash \mathsf{S}(a) \neq 0}\ (\text{S}{\neq}\text{0I})$$

$$\frac{\Gamma \vdash c = 0 \quad \Gamma \vdash a\,\mathsf{N}}{\Gamma \vdash (c\,0?\,a : b) = a}\ (?\text{I1}) \qquad \frac{\Gamma \vdash c \neq 0 \quad \Gamma \vdash b\,\mathsf{N}}{\Gamma \vdash (c\,0?\,a : b) = b}\ (?\text{I2})$$

$$\frac{\Gamma \vdash a \neq b}{\Gamma \vdash b \neq a}\ (\neq\text{S}) \qquad \frac{\Gamma \vdash a \neq b}{\Gamma \vdash \mathsf{S}(a) \neq \mathsf{S}(b)}\ (\neq\text{SI})$$

$$\frac{\Gamma \vdash \mathsf{S}(a) \neq \mathsf{S}(b)}{\Gamma \vdash a \neq b}\ (\text{S}{\neq}\text{E}) \qquad \frac{\Gamma \vdash a \quad \Gamma, a \vdash c}{\Gamma \vdash c}\ (\textsc{cut})$$

$$\frac{\Gamma' \vdash \varphi \quad \Gamma' \subseteq \Gamma}{\Gamma \vdash \varphi}\ (\textsc{wk}) \qquad \frac{\Gamma \vdash a\,\mathsf{N} \quad \Gamma \vdash \varphi[0] \quad \Gamma, v_i\,\mathsf{N}, \varphi[v_i] \vdash \varphi[\mathsf{S}(v_i)]}{\Gamma \vdash \varphi[a]}\ (\textsc{Ind})$$

$$\frac{\Gamma \vdash a_0\,\mathsf{N} \quad \Gamma \vdash a_1\,\mathsf{N} \quad \Gamma \vdash \varphi[\,D_d\langle a_0, a_1\rangle\,]}{\Gamma \vdash \varphi[\,d(a_0, a_1)\,]}\ (\textsc{app})$$

Termination is not a separate judgment form in BGA. $a\,\mathsf{N}$ abbreviates the formula $a = a$. Almost every rule is an introduction rule, and the only elimination is =E.

The checker never applies a rule forwards. It is given a judgment J and the tail ρ of the proof, and asks whether J could have been produced by some rule from judgments occurring in ρ. Every rule with a premise therefore becomes a search through ρ, and the rules whose premises mention a $\varphi[\cdot]$ become a search through ρ combined with a search for a template.

3.12 `bga_subst_rule`

`rep_vars_T :: tm ⇒ tm ⇒ tm`	replaces every leaf of a term by a given variable
`rep_vars_F :: fm ⇒ tm ⇒ fm`	the same on both sides of a formula
`check_template`	searches for a template below a bound
`find_phi`	searches ρ for the premise $\Gamma \vdash \varphi[a]$
`find_eq`	searches ρ for the equation $\Gamma \vdash a = b$
`check_subst :: jdg ⇒ pf ⇒ o`	the rule =E as a whole

The substitution rule is the one place where the checker has to invent something. To justify $\Gamma \vdash \varphi[b]$ it must produce three things: a premise $\Gamma \vdash \varphi[a]$ somewhere in ρ, an equation

$\Gamma \vdash a = b$ somewhere in ρ, and a *template* p, a formula with a designated variable v_i, such that

$$p[v_i \mapsto a] = \varphi[a] \qquad \text{and} \qquad p[v_i \mapsto b] = \varphi[b].$$

The template records which occurrences of a are being rewritten. Nothing in the conclusion says what it is, so it is searched for.

Choosing the variable. The designated variable must not already occur in the judgment, or substituting into the template would disturb something else. The index chosen is $J + 1$, where J is the code of the whole judgment. This is fresh because a variable v_j is encoded as $\mathsf{pack}_T(\mathsf{T_VAR}, j)$, which is a pair with second component j, and pairing and packing are monotone in that component, so $j \le J$ for every variable occurring anywhere in J. Monotonicity of `pack` in §3.6 is assumed for this argument.

Bounding the search. Templates are enumerated downwards from $\mathsf{rep_vars}_F(\varphi[b], J+1)$, the formula obtained by replacing every leaf of the conclusion by v_{J+1}:

$$\mathsf{rep_vars}_T(t,i) = \begin{cases} v_i & t = v_j \text{ or } t = 0 \\ \mathsf{S}(\mathsf{rep_vars}_T(a,i)) & t = \mathsf{S}(a) \\ \mathsf{P}(\mathsf{rep_vars}_T(a,i)) & t = \mathsf{P}(a) \\ (\mathsf{rep_vars}_T(c,i)\ 0?\ \mathsf{rep_vars}_T(a,i) : \mathsf{rep_vars}_T(b,i)) & t = (c\,0?\,a : b) \\ d_j(\mathsf{rep_vars}_T(a,i), \mathsf{rep_vars}_T(b,i)) & t = d_j(a,b) \end{cases}$$

This is the template that abstracts *every* leaf at once. Its code bounds the code of every other candidate with the same constructor skeleton, since such a candidate differs from it only by leaving some leaves unabstracted, and a leaf is numerically smaller than v_{J+1}. `check_template` then counts p down from that bound to zero and tests each value:

$$\mathsf{check_template}(f, \varphi, a, b, p, i) \iff \exists q \le p.\ q[v_i \mapsto a] = \varphi \ \wedge\ q[v_i \mapsto b] = f.$$

```
  fixes check_template :: "fm ⇒ fm ⇒ tm ⇒ tm ⇒ fm ⇒ tm ⇒ o"
(* Counts down p from max_bound to 0.
     i is fixed to a fresh variable (e.g., f+1)
  f - formula we are trying to see if valid
  phi - premise formula
  p - template formula
  i - id of variable acting as a place holder in p
  a=b - equality being considered
 *)
  assumes check_template_def: "check_template f phi a b p i :=
    if subst_F p i a = phi ∧ subst_F p i b = f then True
    else if p > 0 = 1 then check_template f phi a b (p - 1) i
    else False"
```

```
  fixes rep_vars_T     :: "tm ⇒ tm ⇒ tm"
(* rep_vars_T t i: Replaces all leaves in term t with variable i *)
  assumes rep_vars_T_def: "rep_vars_T t i :=
    if tag_T t = T_VAR then pack_T T_VAR i
    else if tag_T t = T_ZERO then pack_T T_VAR i
    else if tag_T t = T_SUC then pack_T T_SUC (rep_vars_T (load_T t) i)
    else if tag_T t = T_PRED then pack_T T_PRED (rep_vars_T (load_T t) i)
    else if tag_T t = T_IFZ then
      pack_T T_IFZ ⟨rep_vars_T (cpx (load_T t)) i,
               ⟨rep_vars_T (cpx (cpy (load_T t))) i,
                rep_vars_T (cpy (cpy (load_T t))) i⟩⟩
```

Remark 3.2 (The bound matters for completeness only)**.** Soundness of the checker does not depend on the bound being correct. `check_template_sound_fuel` assumes that the search reported success and extracts the witness q it found; a bound that was too small would make the checker reject proofs it ought to accept, which is a completeness failure, and would leave soundness untouched. Nothing in this development proves completeness.

Searching the proof. With the template search in place, the two remaining searches walk the proof list. In mathematical notation, writing $K \in \rho$ for membership in the proof list,

$$\begin{aligned}
\mathsf{find_phi}(J,a,b,\pi) &\iff \exists K \in \pi.\ \mathsf{hyp}(K) = \mathsf{hyp}(J) \ \wedge\ \mathsf{check_template}(\mathsf{conc}(J),\mathsf{conc}(K),a,b,\beta_J,J+1),\\
\mathsf{find_eq}(J,\rho,\pi) &\iff \exists K \in \pi.\ \mathsf{hyp}(K) = \mathsf{hyp}(J) \ \wedge\ \mathsf{conc}(K) \text{ is } (a=b) \ \wedge\ \mathsf{find_phi}(J,a,b,\rho),\\
\mathsf{check_subst}(J,\rho) &\iff \mathsf{find_eq}(J,\rho,\rho),
\end{aligned}$$

where $\beta_J = \mathsf{rep_vars}_F(\mathsf{conc}(J), J+1)$ is the template bound. Both searches carry two proof arguments. The one named `ptr` is the part still to be scanned and shrinks on each step, and the one named `rest` is the whole tail of the proof and stays fixed, because the premise being looked for may sit anywhere in it. The hypothesis test $\mathsf{hyp}(K) = \mathsf{hyp}(J)$ is what keeps the rule from silently changing the context.

```
  fixes find_phi       :: "jdg ⇒ tm ⇒ tm ⇒ pf ⇒ o"
(* Scans the proof list (ptr) for a valid premise J_phi = ⟨Γ, φ⟩
   such that Γ matches J's context, and p[v_(f+1) ↦ a] = φ and p[v_(f+1) ↦ b] = f *)
  assumes find_phi_def: "find_phi J a b ptr :=
    if ptr = Nil then False
    else if hyp_of (list_hd ptr) = hyp_of J ∧
            check_template (conc_of J) (conc_of (list_hd ptr)) a b
                           (rep_vars_F (conc_of J) (J + 1)) (J + 1) then True
    else find_phi J a b (list_tl ptr)"
```

```
  fixes find_eq        :: "jdg ⇒ pf ⇒ pf ⇒ o"
(* Scans the proof list (ptr) for an equality judgment J_eq = ⟨Γ, a = b⟩
   such that Γ matches J's context, and calls find_phi *)
  assumes find_eq_def: "find_eq J rest ptr :=
    if ptr = Nil then False
    else if hyp_of (list_hd ptr) = hyp_of J ∧ tag_F (conc_of (list_hd ptr)) = F_EQ then
      if find_phi J (cpx (load_F (conc_of (list_hd ptr)))) (cpy (load_F (conc_of (list_hd
       ↪  ptr)))) rest then True
      else find_eq J rest (list_tl ptr)
    else find_eq J rest (list_tl ptr)"
  fixes check_subst    :: "jdg ⇒ pf ⇒ o"

(* checking the substitution rule *)
assumes check_subst_def: "check_subst J rest := find_eq J rest rest"
```

3.13 `bga_eq_rule`

`check_eq_rules :: hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o`	the seven rules concluding an equation
`check_neq_rules :: hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o`	the four rules concluding a disequation

Both take the two sides and their tags as separate arguments, already projected out of the conclusion by `valid_step`, so that the branches can test tags without recomputing them.

```
  fixes check_eq_rules  :: "hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o"
(* Equality Rules
1. Γ ⊢ 0 N
2. Γ ⊢ a=b ⟹ Γ ⊢ b=a
3. Γ ⊢ a=b ⟹ Γ ⊢ S(a)=S(b)
4. Γ ⊢ S(a)=S(b) ⟹ Γ ⊢ a=b
5. Γ ⊢ a N ⟹ Γ ⊢ P(S(a))=a
6. Γ ⊢ c≠0 ⟹ Γ ⊢ b N ⟹ Γ ⊢ (c 0? a: b)=b
7. Γ ⊢ c=0 ⟹ Γ ⊢ a N ⟹ Γ ⊢ (c 0? a: b)=a
*)
  assumes check_eq_rules_def: "check_eq_rules G lhs rhs tg_L tg_R rest :=
    if lhs = pack_T T_ZERO 0 ∧ rhs = pack_T T_ZERO 0 then True
    else if mem (G ⊩ pack_F F_EQ ⟨rhs, lhs⟩) rest then True
    else if tg_L = T_SUC ∧ tg_R = T_SUC ∧
            mem (G ⊩ pack_F F_EQ ⟨load_T lhs, load_T rhs⟩) rest then True
    else if mem (G ⊩ pack_F F_EQ ⟨pack_T T_SUC lhs, pack_T T_SUC rhs⟩) rest then True
    else if tg_L = T_PRED ∧ tag_T (load_T lhs) = T_SUC ∧
            (load_T (load_T lhs)) = rhs ∧
            mem (G ⊩ pack_F F_EQ ⟨rhs, rhs⟩) rest then True
    else if tg_L = T_IFZ ∧ rhs = cpy (cpy (load_T lhs)) ∧
            mem (G ⊩ pack_F F_NEQ ⟨cpx (load_T lhs), pack_T T_ZERO 0⟩) rest ∧
            mem (G ⊩ pack_F F_EQ ⟨rhs, rhs⟩) rest then True
    else if tg_L = T_IFZ ∧ rhs = cpx (cpy (load_T lhs)) ∧
            mem (G ⊩ pack_F F_EQ ⟨cpx (load_T lhs), pack_T T_ZERO 0⟩) rest ∧
            mem (G ⊩ pack_F F_EQ ⟨rhs, rhs⟩) rest then True
    else False"
```

The seven branches are, in order, 0I, =S, S=I, S=E, PI, ?I2 and ?I1. Each is a membership test on `rest`, so a judgment is justified only by judgments occurring later in the list, which is what makes `check_list` a well-founded recursion. Reading the fifth branch as an example: the conclusion is $\Gamma \vdash \mathsf{P}(u) = b$, the branch checks that u is $\mathsf{S}(b)$ by inspecting tags and payloads, and it requires $\Gamma \vdash b = b$ to appear in the proof, which is the habeas quid premise $b\ \mathsf{N}$ of PI.

```
  fixes check_neq_rules :: "hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o"
(* Not-Equality Rules
1. Γ ⊢ a≠b ⟹ Γ ⊢ b≠a
2. Γ ⊢ a N ⟹ Γ ⊢ S(a)≠0
3. Γ ⊢ a≠b ⟹ Γ ⊢ S(a)≠S(b)
4. Γ ⊢ S(a)≠S(b) ⟹ Γ ⊢ a≠b
*)
  assumes check_neq_rules_def: "check_neq_rules G lhs rhs tg_L tg_R rest :=
    if mem (G ⊩ pack_F F_NEQ ⟨rhs, lhs⟩) rest then True
    else if tg_L = T_SUC ∧ rhs = pack_T T_ZERO 0 ∧
            mem (G ⊩ pack_F F_EQ ⟨load_T lhs, load_T lhs⟩) rest then True
    else if tg_L = T_SUC ∧ tg_R = T_SUC ∧
            mem (G ⊩ pack_F F_NEQ ⟨load_T lhs, load_T rhs⟩) rest then True
    else if mem (G ⊩ pack_F F_NEQ ⟨pack_T T_SUC lhs, pack_T T_SUC rhs⟩) rest then True
    else False"
```

and dually ≠S, S≠0I, ≠SI and S≠E.

3.14 `bga_ind_rule`

`check_ind_template`	tests one candidate template for the induction rule
`find_ind_base`	searches ρ for the base case
`check_ind :: jdg ⇒ pf ⇒ o`	the rule IND as a whole

Induction needs the same kind of template, and three premises rather than one. To justify

$\Gamma \vdash \varphi[a]$ it looks for a template q with

$$q[v_i \mapsto a] = \varphi[a], \qquad q[v_i \mapsto 0] \text{ present in } \rho, \qquad (\, v_i = v_i,\ q,\ \Gamma \vdash q[v_i \mapsto \mathsf{S}(v_i)]\,) \text{ present in } \rho,$$

that is,

$$\mathsf{check_ind_template}(f, \varphi, a, \Gamma, p, i, \rho) \iff \exists q \le p.\ q[v_i \mapsto a] = f \ \wedge\ q[v_i \mapsto 0] = \varphi \ \wedge\ (v_i = v_i,\, q,\, \Gamma \vdash q[v_i \mapsto \mathsf{S}(v_i)]) \in \rho.$$

```
fixes check_ind_template :: "fm ⇒ fm ⇒ tm ⇒ hyp ⇒ fm ⇒ tm ⇒ pf ⇒ o"
(*counts down 'p', testing if it is the valid induction template
checks for a p such that:
p[v_i → a] = φ
Γ ⊢ p[v_i → 0] in proof
[v_i = v_i, p]+Γ ⊢ p[v_i → S(v_i)] in proof

*)
  assumes check_ind_template_def: "check_ind_template f phi a G p i rest :=
    if subst_F p i a = f ∧
       subst_F p i (pack_T T_ZERO 0) = phi ∧
       mem (pack_F F_EQ ⟨pack_T T_VAR i, pack_T T_VAR i) ▹ p ▹ G ⊩
                subst_F p i (pack_T T_SUC (pack_T T_VAR i))) rest
    then True
    else if p > 0 = 1 then check_ind_template f phi a G (p - 1) i rest
    else False"
```

```
  fixes check_ind       :: "jdg ⇒ pf ⇒ o"
  (* check_ind triggers the search if the Habeas Quid premise Γ ⊢ a N (encoded as a=a) exists
   ↪  *)
  assumes check_ind_def: "check_ind J rest :=
    if fresh_H (J + 1) (hyp_of J) ∧ mem (hyp_of J ⊩ pack_F F_EQ ⟨cpx (load_F (conc_of J)), cpx
     ↪  (load_F (conc_of J)))) rest then
       find_ind_base J (cpx (load_F (conc_of J))) rest rest
    else False"
```

`check_ind` guards the search with two conditions. The habeas quid premise $\Gamma \vdash a\,\mathsf{N}$, encoded as the judgment $\Gamma \vdash a = a$, must be in the proof, and $\mathrm{fresh}_H(J+1, \Gamma)$ must hold, so that the variable used to build the template does not occur in the hypotheses. The step case sought in the proof carries both the induction hypothesis q and the termination assumption $v_i = v_i$ on the induction variable, which is what the semantic induction lemma of §4 consumes. `find_ind_base` scans ρ for the base case in the same style as `find_phi`.

3.15 `bga_struct_rule`

`find_cut`	searches ρ for the cut formula
`check_cut :: jdg ⇒ pf ⇒ o`	the rule CUT
`find_struct`	searches ρ for the weakened judgment
`check_struct :: jdg ⇒ pf ⇒ o`	the rule WK

```
locale bga_struct_rule = bga_encoding +


fixes find_cut    :: "jdg ⇒ pf ⇒ pf ⇒ o"
fixes check_cut   :: "jdg ⇒ pf ⇒ o"

  (* Cut Rule:
  Γ ⊢ a ⟹ a + Γ ⊢ c ⟹ Γ ⊢ c
```

```
  *)
  assumes find_cut_def: "find_cut J rest ptr :=
    if ptr = Nil then False
    else if hyp_of (list_hd ptr) = hyp_of J then
      if mem ((conc_of (list_hd ptr)) ▷ (hyp_of J) ⊩ (conc_of J)) rest then True
      else find_cut J rest (list_tl ptr)
    else find_cut J rest (list_tl ptr)"

  assumes check_cut_def: "check_cut J rest := find_cut J rest rest"

  fixes find_struct :: "jdg ⇒ hyp ⇒ pf ⇒ o"
  assumes find_struct_def: "find_struct J G ptr :=
    if ptr = Nil then False
    else if conc_of (list_hd ptr) = conc_of J ∧ subset (hyp_of (list_hd ptr)) G then True
    else find_struct J G (list_tl ptr)"

  fixes check_struct :: "jdg ⇒ pf ⇒ o"
  assumes check_struct_def: "check_struct J rest := find_struct J (hyp_of J) rest"
```

$$\mathsf{check_cut}(J,\rho) \iff \exists K \in \rho.\ \mathsf{hyp}(K) = \mathsf{hyp}(J) \ \wedge\ (\mathsf{conc}(K),\, \mathsf{hyp}(J) \vdash \mathsf{conc}(J)) \in \rho,$$
$$\mathsf{check_struct}(J,\rho) \iff \exists K \in \rho.\ \mathsf{conc}(K) = \mathsf{conc}(J) \ \wedge\ \mathsf{hyp}(K) \subseteq \mathsf{hyp}(J).$$

The hypothesis rule is not a separate function. It is the first branch of `valid_step`, the test `mem (conc_of J) (hyp_of J)`.

3.16 `bga_app_rule`

`app_try`	tests one triple (d, x, y)
`app_y`	counts y down
`app_x`	counts x down
`app_d`	counts the definition index d down
`check_app :: jdg ⇒ pf ⇒ o`	the rule APP as a whole

```
fixes app_try :: "jdg ⇒ num ⇒ num ⇒ num ⇒ pf ⇒ o"
  assumes app_try_def: "app_try J d x y rest :=
    dfn_is d 2 (nth d dfns) ∧
    mem (hyp_of J ⊩ pack_F F_EQ ⟨x, x⟩) rest ∧
    mem (hyp_of J ⊩ pack_F F_EQ ⟨y, y⟩) rest ∧
    find_phi J (subst_body (nth d dfns) x y) (pack_T T_APP ⟨d, ⟨x, y⟩⟩) rest"

(* Tries to see if application is satisfied for some y*)
fixes app_y :: "jdg ⇒ num ⇒ num ⇒ num ⇒ pf ⇒ o"
  assumes app_y_def: "app_y J d x y rest :=
    if app_try J d x y rest then True
    else if y > 0 = 1 then app_y J d x (y - 1) rest
    else False"

(* Tries to see if application is satisfied for some x, y with y bounded by f (where J= Γ⊢f) *)
  fixes app_x :: "jdg ⇒ num ⇒ num ⇒ pf ⇒ o"
  assumes app_x_def: "app_x J d x rest :=
    if app_y J d x (conc_of J) rest then True
    else if x > 0 = 1 then app_x J d (x - 1) rest
    else False"

(*Given a J and an upperbound for def, sees if application can be satisfied by any d,x,y
 ↪  triplet *)
  fixes app_d :: "jdg ⇒ num ⇒ pf ⇒ o"
  assumes app_d_def: "app_d J d rest :=
```

```
    if d < len dfns = 1 then
      (if app_x J d (conc_of J) rest then True
       else if d > 0 = 1 then app_d J (d - 1) rest else False)
    else
      (if d > 0 = 1 then app_d J (d - 1) rest else False)"

  fixes check_app :: "jdg ⇒ pf ⇒ o"
  assumes check_app_def: "check_app J rest := app_d J (len dfns - 1) rest"
```

Application unfolds a definition. To justify $\Gamma \vdash \varphi[d(x,y)]$ the checker needs $\Gamma \vdash x\,\mathsf{N}$, $\Gamma \vdash y\,\mathsf{N}$ and a premise $\Gamma \vdash \varphi[\,D_d[v_0 \mapsto x, v_1 \mapsto y]\,]$:

$$\mathsf{app_try}(J,d,x,y,\rho) \iff \mathsf{dfn_is}(d,2,D_d) \;\wedge\; (\mathsf{hyp}(J) \vdash x = x) \in \rho \;\wedge\; (\mathsf{hyp}(J) \vdash y = y) \in \rho \\ \wedge\; \mathsf{find_phi}(J,\; D_d[v_0 \mapsto x, v_1 \mapsto y],\; d(x,y),\; \rho).$$

Note that `find_phi` is reused here with the two sides of the rewrite being the unfolded body and the application, so unfolding a definition is treated as one more instance of rewriting inside a template.

None of d, x and y is determined by the conclusion, so all three are enumerated:

$$\mathsf{check_app}(J,\rho) \iff \exists d < |D|.\;\; \exists x \le \mathsf{conc}(J).\;\; \exists y \le \mathsf{conc}(J).\;\; \mathsf{app_try}(J,d,x,y,\rho).$$

The three nested countdowns `app_d`, `app_x` and `app_y` are that triple search written as recursion, since QGA has no bounded quantifier. The bound $\mathsf{conc}(J)$ on the arguments is the same kind of bound as in §3.12: the code of a subterm is at most the code of the formula containing it, and as before it is completeness rather than soundness that depends on it.

3.17 `bga_proof_check`

`valid_step :: jdg ⇒ pf ⇒ o`	"J follows by one rule from judgments in ρ"
`check_list :: pf ⇒ o`	every entry follows from the entries after it
`is_valid_proof :: pf ⇒ jdg ⇒ o`	"π is a valid proof whose head is J"

```
fixes valid_step :: "jdg ⇒ pf ⇒ o"

(* The main step check
1. 1 hyp Rule
2. 1 Cut Rule
3. 1 Subst Rule
4. 1 Induction Rule
5. 7 Equality Rules
6. 4 Not-equality rules
7. 1 app2I rule
8. 1 wk1 rule
sub1
 *)
  assumes valid_step_def: "valid_step J rest :=
    if mem (conc_of J) (hyp_of J) then True
    else if check_cut J rest then True
    else if check_subst J rest then True
    else if check_ind J rest then True
    else if check_app J rest then True
    else if check_struct J rest then True
    else if tag_F (conc_of J) = F_EQ then
      check_eq_rules (hyp_of J) (cpx (load_F (conc_of J))) (cpy (load_F (conc_of J)))
                     (tag_T (cpx (load_F (conc_of J)))) (tag_T (cpy (load_F (conc_of J)))) rest
    else
      check_neq_rules (hyp_of J) (cpx (load_F (conc_of J))) (cpy (load_F (conc_of J)))
```

```
                    (tag_T (cpx (load_F (conc_of J)))) (tag_T (cpy (load_F (conc_of J)))) rest"
```

`valid_step` is the disjunction of the branches, tried in order: membership for HYP, then cut, substitution, induction, application and weakening, and finally the equality or disequality rules according to the tag of the conclusion. Written as a formula,

$$\begin{aligned}\mathsf{valid_step}(J,\rho) \iff\ & \mathsf{conc}(J) \in \mathsf{hyp}(J) \ \vee\ \mathsf{check_cut}(J,\rho) \ \vee\ \mathsf{check_subst}(J,\rho) \\ & \vee\ \mathsf{check_ind}(J,\rho) \ \vee\ \mathsf{check_app}(J,\rho) \ \vee\ \mathsf{check_struct}(J,\rho) \\ & \vee\ (\mathsf{tag}\ \mathsf{conc}(J) = \mathsf{F_EQ}\ ?\ \mathsf{check_eq_rules}(\dots)\ :\ \mathsf{check_neq_rules}(\dots)).\end{aligned}$$

The order is not significant for soundness. It is a sequence of nested conditionals rather than a disjunction because each branch has to be evaluated only when the earlier ones fail.

```
fixes check_list :: "pf ⇒ o"
assumes check_list_def: "check_list pf :=
    if pf = Nil then True
    else if valid_step (list_hd pf) (list_tl pf) then check_list (list_tl pf)
    else False"
```

```
assumes is_valid_proof_def: "is_valid_proof pf J :=
    if pf = Nil then False
    else if list_hd pf = J then check_list pf
    else False"
```

A proof is valid when every entry follows from the entries after it, and it proves J when additionally its head is J:

$$\mathsf{check_list}(\pi) \iff \pi = \mathsf{Nil} \ \vee\ (\mathsf{valid_step}(\mathsf{hd}\,\pi, \mathsf{tl}\,\pi) \wedge \mathsf{check_list}(\mathsf{tl}\,\pi)),$$

$$\mathsf{is_valid_proof}(\pi, J) \iff \pi \neq \mathsf{Nil} \ \wedge\ \mathsf{hd}\,\pi = J \ \wedge\ \mathsf{check_list}(\pi).$$

Reading the list head first, with each entry justified by its tail, is what makes the recursion terminate and what the soundness induction of §4.4 follows.

4 The consistency proof

This section follows the argument from the evaluator up to the final theorem. The intent is to make the shape of the proof and its load-bearing lemmas visible.

4.1 Shape of the argument

Everything reduces to one implication, called the soundness bridge:

> if `p` checks as a proof of `J`, and the hypotheses of `J` are satisfied by an assignment `A`, then the conclusion of `J` is satisfied by `A`.

Given it, §3.4 closes the argument in half a page. Proving it requires four things, and they are the subsections that follow.

1. The step-indexed evaluator determines at most one value, so that `evals` behaves as a partial function and satisfaction is well behaved.
2. Every branch of `valid_step` preserves satisfaction, assuming that every judgment in the tail of the proof does.

3. That local statement can be lifted along the proof list by induction.

4. Satisfaction of an encoded equation and of an encoded disequality can be unpacked into statements about values.

A remark on technique before the details. QGA induction instantiates a schematic predicate `Q`, so a statement can only be generalized before inducting if the generalization is part of the object-level formula. Several proofs in this file are therefore stated internally in the form `∀u. ∀A2. …`, or as nested implications `… ⟶ (… ⟶ …)`, and unpacked afterwards with `forallE` and `implE`. `eval_fuel_N` is the first instance: the induction on fuel runs on `∀u. ∀A2. eval_fuel k u A2 N` and the desired statement is recovered by two instantiations.

Nesting an implication inside a motive has a price. ⟶ introduction requires its antecedent to be grounded, so every antecedent of such a motive needs a groundedness lemma. The checkers all have one. Satisfaction does not, and §3.9 is what supplies it.

4.2 The evaluator is a partial function

The fuel evaluator is total as a function into `num`, which is the statement `eval_fuel_N: ⟦k N; t N; A N⟧ ⟹ eval_fuel k t A N`, and 0 is its "no result" value. Success is preserved when the budget grows.

```
lemma eval_fuel_success_add:
  assumes k: "k N" and n: "n N" and t: "t N" and A: "A N" and result: "eval_fuel k t A = S r"
  shows "eval_fuel (k + n) t A = S r"
```

This is proved by induction on the increment, from the one-step version `eval_fuel_success_suc`. Two successful runs with different budgets agree, since both can be raised to the common budget `k + l`.

```
lemma eval_fuel_success_unique:
  assumes k: "k N" and l: "l N" and t: "t N" and A: "A N"
      and left: "eval_fuel k t A = S r" and right: "eval_fuel l t A = S q"
  shows "r = q"
```

The existential form follows, and it is the assumption `evals_det` that the abstract locale of §3.4 demanded.

```
lemma evals_functional:
  assumes t: "t N" and A: "A N" and left: "evals t A r" and right: "evals t A q"
  shows "r = q"
```

The remaining facts about the evaluator are one lemma per constructor, in two directions each: from a successful run one can read off the value of the subterms (`eval_fuel_suc_value`, `eval_fuel_pred_value`, `eval_fuel_ifz_zero_value`, `eval_fuel_app_value`), and from successful runs of the subterms one can build a successful run of the whole (`eval_fuel_sucI` and its relatives). Together they let `evals` be used as an inductively defined big-step relation, which is how the rule soundness proofs consume it.

4.3 Soundness of a single step

The central local statement is the following. It says that a checked step is sound relative to the tail of the proof, and it is what the induction of §4.4 will feed itself.

```
lemma valid_step_sound_fuel:
  assumes J: "J N" and rest: "rest N" and A: "A N"
      and vs: "valid_step J rest"
      and prev:
        "⋀K A2. A2 N ⟹ K N ⟹ mem K rest ⟹
          sat_hyp_fuel (hyp_of K) A2 ⟹
          sat_fuel (conc_of K) A2"
      and satG: "sat_hyp_fuel (hyp_of J) A"
  shows "sat_fuel (conc_of J) A"
```

Its proof is a case analysis over the branches of `valid_step`, using `cases_bool` on each guard in turn, with one lemma per branch. All the branch lemmas have the same premises as above and the same conclusion; they differ only in which checker they assume.

Branch	**Lemma**
hypothesis	inline, from `sat_hyp_fuel_mem`
cut	`check_cut_sound_fuel`
substitution of equals	`check_subst_sound_fuel`
induction	`check_ind_sound_fuel`
application	`check_app_sound_fuel`
weakening	`check_struct_sound_fuel`
the seven equality rules	`check_eq_rules_sound_fuel`
the four disequality rules	`check_neq_rules_sound_fuel`

Two of these carry the real content.

The template lemma. Substitution of equals, application and induction all reduce to the same semantic fact: if a template `p` instantiated at `a` is satisfied, and `a` and `b` evaluate to a common value, then the same template instantiated at `b` is satisfied.

```
lemma template_instance_sound_fuel:
  assumes p: "p N" and i: "i N" and a: "a N" and b: "b N"
      and phi: "phi N" and f: "f N" and A: "A N" and v: "v N"
      and pa: "subst_F p i a = phi"
      and pb: "subst_F p i b = f"
      and sphi: "sat_fuel phi A"
      and eva: "evals a A v"
      and evb: "evals b A v"
  shows "sat_fuel f A"
```

It rests on a pair of substitution lemmas that relate syntactic substitution to updating the assignment.

```
lemma sat_fuel_subst_FD:
  assumes f: "f N" and i: "i N" and s: "s N" and A: "A N" and v: "v N"
      and evs: "evals s A v" and sat: "sat_fuel (subst_F f i s) A"
  shows "sat_fuel f (asn_put A i v)"
```

```
lemma sat_fuel_subst_FI:
  assumes f: "f N" and i: "i N" and s: "s N" and A: "A N" and v: "v N"
      and evs: "evals s A v" and sat: "sat_fuel f (asn_put A i v)"
  shows "sat_fuel (subst_F f i s) A"
```

Given `a` and `b` that both evaluate to `v`, the first pushes satisfaction of `p[i ↦ a]` under `A` down to satisfaction of `p` under `asn_put A i v`, and the second lifts it back up to `p[i ↦ b]`. The search

functions are then sound because they report success only when such a template exists, which is the content of `check_template_sound_fuel` and of

```
lemma find_phi_sound_fuel:
  assumes J: "J N" and a: "a N" and b: "b N"
      and rest: "rest N" and ptr: "ptr N" and A: "A N"
      and sub: "subset ptr rest"
      and fp: "find_phi J a b ptr"
      and prev: "⋀K. K N ⟹ mem K rest ⟹
        sat_hyp_fuel (hyp_of K) A ⟹
        sat_fuel (conc_of K) A"
      and satG: "sat_hyp_fuel (hyp_of J) A"
      and tr: "⋀q. q N ⟹ evals a A q ⟹ evals b A q"
  shows "sat_fuel (conc_of J) A"
```

The premise `prev` is the induction hypothesis of §4.4 in the form the search needs: every judgment occurring in `rest` is sound. The premise `sub` records that the pointer walks a sublist of `rest`, which is what keeps the recursion inside the part of the proof already known to be sound. Application is the same lemma applied to the template $\varphi[\,\mathrm{body}_d[v_0 \mapsto x, v_1 \mapsto y]\,]$, with the two habeas quid premises supplying the values that `subst_body` needs.

The induction rule. Soundness of the induction rule is the one place where a semantic induction is performed, over the value of the term the rule inducts on.

```
lemma nat_ind_sound_put_fuel:
  assumes p: "p N" and i: "i N" and a: "a N"
      and G: "G N" and A: "A N" and n: "n N"
      and fresh: "fresh_H i G"
      and satG: "sat_hyp_fuel G A"
      and base: "sat_fuel (subst_F p i (pack_T T_ZERO 0)) A"
      and step:
        "⋀m. m N ⟹
          sat_hyp_fuel
            (pack_F F_EQ
               ⟨pack_T T_VAR i, pack_T T_VAR i⟩
               ▹ p ▹ G)
            (asn_put A i m) ⟹
          sat_fuel
            (subst_F p i
              (pack_T T_SUC (pack_T T_VAR i)))
            (asn_put A i m)"
      and eva: "evals a A n"
  shows "sat_fuel (subst_F p i a) A"
```

Read semantically: if the hypothesis list is satisfied, the base instance $p[v_i \mapsto 0]$ is satisfied, and the step is satisfied for every value `m` put into the assignment at index `i`, then the instance at `a` is satisfied whenever `a` evaluates to `n`. The proof establishes `⋀m. m N ⟹ sat_fuel p (asn_put A i m)` by QGA induction on `m`, with the base case supplied by `sat_fuel_subst_FD` applied to the encoded zero, and then instantiates it at `n`. The freshness premise `fresh_H i G` is what allows the assignment to be modified at `i` without disturbing satisfaction of the hypotheses. `check_ind_sound_fuel` then supplies the three premises from the corresponding judgments found in the proof list.

4.4 From steps to proofs

The lifting step is a dedicated induction principle over proof lists.

```
lemma check_list_induct_N:
  assumes pf: "pf N"
      and base: "⋀J A. A N ⟹ check_list Nil ⟹ J N ⟹ mem J Nil ⟹
                   sat_hyp (hyp_of J) A ⟹ sat (conc_of J) A"
      and step: "⋀h t. h N ⟹ t N ⟹
                   (⋀J A. A N ⟹ check_list t ⟹ J N ⟹ mem J t ⟹
                      sat_hyp (hyp_of J) A ⟹ sat (conc_of J) A) ⟹
                   (⋀J A. A N ⟹ check_list (Cons h t) ⟹
                      J N ⟹ mem J (Cons h t) ⟹
                      sat_hyp (hyp_of J) A ⟹ sat (conc_of J) A)"
  shows "⋀J A. A N ⟹ check_list pf ⟹ J N ⟹ mem J pf ⟹
                sat_hyp (hyp_of J) A ⟹ sat (conc_of J) A"
```

It is stated for `sat_fuel` and `sat_hyp_fuel`. Its only caller instantiates it at those two, and the indexed predicates it uses internally are defined in terms of them.

Internally it is proved by `list_induct` on a statement of the form

```
∀A. ∀K. ∀k. check_list pf ⟶ (mem K pf ⟶ (sat_hyp_at k (hyp_of K) A ⟶ sat_fuel
                                  (conc_of K) A)).
```

The predicate being inducted on has to be a single formula, so the generalizations over the assignment, over the judgment and over the budget sit inside it as QGA quantifiers.

The three antecedents are why the budget is explicit. Each one has to be grounded before ⟶ can be introduced, and `check_list_bool`, `mem_bool` and `sat_hyp_at_bool` supply exactly that at the three levels. The conclusion needs no such treatment, since ⟶ introduction constrains only the antecedent, which is why `sat_fuel` appears there unindexed. With `sat_hyp_fuel` in the antecedent instead, the introduction would be blocked.

The indexed predicate never leaves the proof. The statement of the lemma is in `sat_hyp_fuel` and `sat_fuel`, and the bridges of §3.9 convert at the boundary. `sat_hyp_atD` runs downward at the two points where the induction hands control to `base` and `step`, both of which are stated in the unindexed predicates. `sat_hyp_at_exI` runs upward twice, once in the tail case where the induction hypothesis needs a budget for the hypotheses of another judgment, and once at the end where the lemma's own assumption has to produce one.

Instantiating the principle and discharging the step case with `valid_step_sound_fuel` gives soundness for every judgment occurring in a checked list.

```
lemma check_list_sound_fuel_N:
  assumes pf: "pf N"
  shows "⋀J A. A N ⟹ check_list pf ⟹
    J N ⟹ mem J pf ⟹
    sat_hyp_fuel (hyp_of J) A ⟹
    sat_fuel (conc_of J) A"
```

Finally `is_valid_proof` is unfolded. Its two guards are decided with `cases_bool`, the head of the proof is shown to be a member of it by `cons_reconstr` and `mem_cons_head`, and the previous lemma applies.

```
lemma soundness_bridge_fuel:
  assumes vp: "is_valid_proof p J"
      and satG: "sat_hyp_fuel (hyp_of J) A"
      and AN: "A N"
  shows "sat_fuel (conc_of J) A"
```

4.5 Consistency

The two formula constructors are defined, and their satisfaction is unpacked.

```
definition mk_eq :: "num ⇒ num ⇒ num" where
    "mk_eq a b ≡ pack_F 0 ⟨a, b⟩"

definition mk_neq :: "num ⇒ num ⇒ num" where
    "mk_neq a b ≡ pack_F 1 ⟨a, b⟩"
```

```
lemma sat_fuel_mk_eqE:
  assumes a: "a N" and b: "b N" and sat: "sat_fuel (mk_eq a b) A"
  obtains q where "q N" and "evals a A q" and "evals b A q"
```

```
lemma sat_fuel_mk_neqE:
  assumes a: "a N" and b: "b N" and sat: "sat_fuel (mk_neq a b) A"
  obtains x y where "x N" and "y N" and "evals a A x" and "evals b A y" and "x ≠ y"
```

Both unpacking lemmas go through `tag_pack_F` and `load_pack_F` to recover the two sides from the code, and then through the definition of `sat_fuel`. Groundedness of proof checking, the last assumption of the skeleton, is `proof_is_bool`, obtained from `check_list_bool` by list induction and from a chain of `condTB` steps over the branches of `valid_step`.

With these in place, `bga_fuller`, the union of the step-indexed semantics with the proof checker, is declared a sublocale of `consistent`.

```
sublocale consistent mk_eq mk_neq dfns is_valid_proof evals sat_fuel sat_hyp_fuel
proof (unfold_locales)
  show "⋀a b. a N ⟹ b N ⟹ mk_eq a b N"
    by (rule mk_eq_N')
  show "⋀a b. a N ⟹ b N ⟹ mk_neq a b N"
    by (rule mk_neq_N')
  show "⋀p J. p N ⟹ J N ⟹ is_valid_proof p J B"
    by (rule proof_is_bool)
  show "⋀A. sat_hyp_fuel Nil A"
    by (rule sat_hyp_fuel_nil)
  show "⋀t A r q. t N ⟹ A N ⟹ evals t A r ⟹ evals t A q ⟹ r = q"
    by (rule evals_functional)
  show "⋀a b A R. a N ⟹ b N ⟹ sat_fuel (mk_eq a b) A ⟹
        (⋀q. q N ⟹ evals a A q ⟹ evals b A q ⟹ R) ⟹ R"
    by (rule sat_fuel_mk_eqE)
  show "⋀a b A R. a N ⟹ b N ⟹ sat_fuel (mk_neq a b) A ⟹
        (⋀x y. x N ⟹ y N ⟹ evals a A x ⟹ evals b A y ⟹ x ≠ y ⟹ R) ⟹ R"
    by (rule sat_fuel_mk_neqE)
  show "⋀p J A. is_valid_proof p J ⟹ sat_hyp_fuel (hyp_of J) A ⟹ A N ⟹ sat_fuel (conc_of
   ↪  J) A"
    by (rule soundness_bridge_fuel)
```

Each obligation is discharged by one of the lemmas above: `mk_eq_N'` and `mk_neq_N'` for habeas quid, `proof_is_bool` for groundedness of the checker, `sat_hyp_fuel_nil` for the empty hypothesis list, `evals_functional` for determinism, `sat_fuel_mk_eqE` and `sat_fuel_mk_neqE` for the two unpacking assumptions, and `soundness_bridge_fuel` for soundness. The theorem `syntactically_consistent` of §3.4 becomes available inside `bga_fuller`, and it remains to exhibit an encoding that satisfies the interface.

5 The concrete encoding

`encode.thy` instantiates the interface of §3.6 with a Gödel numbering built from the Cantor pairing of §2.11, supplies concrete recursive definitions for every function the checker is built from, discharges all the assumptions, and instantiates the consistency theorem.

5.1 The encoders

A code is a pair whose first component is the constructor tag and whose second is the payload. Formulas use the pairing directly, and terms compose it with an involution that exchanges 0 and 1.

```
definition swap01 :: "num ⇒ num" where
  "swap01 t ≡ if t = 0 then 1 else if t = 1 then 0 else t"

definition tag_T  :: "tm ⇒ tmtag"        where "tag_T  n    ≡ swap01 (cpx n)"
definition load_T :: "tm ⇒ tm"           where "load_T n    ≡ cpy n"
definition pack_T :: "tmtag ⇒ tm ⇒ tm"   where "pack_T tg L ≡ ⟨swap01 tg, L⟩"
definition tag_F  :: "fm ⇒ fmtag"        where "tag_F  n    ≡ cpx n"
definition load_F :: "fm ⇒ tm"           where "load_F n    ≡ cpy n"
definition pack_F :: "fmtag ⇒ fm ⇒ fm"   where "pack_F tg L ≡ ⟨tg, L⟩"
```

Remark 5.1 (Why `swap01`)**.** The interface requires `tag_T 0 = T_ZERO`, so the code 0 must denote the term 0. With `T_ZERO = 1` and `cpx 0 = 0`, taking `tag_T = cpx` would make the code 0 a variable reference, since `T_VAR = 0`, and the structural-decrease assumption `t ≠ 0 ⟹ load_T t < t = 1` would then have no atomic base to stop at. Composing with `swap01` on both the packing and the unpacking side relabels those two tags and leaves everything else alone, and `swap01 (swap01 t) = t` makes the composition cancel. Formulas need no adjustment, since `F_EQ = 0` already.

```
lemma swap01_0 [simp]: "swap01 0 = 1"
  unfolding swap01_def by (cases bool: "(0::num) = 0", simp+)

lemma swap01_N [simp, auto]: "t N ⟹ swap01 t N"
  unfolding swap01_def
  apply (cases bool: "t = 0", simp+)
  done
```

`swap01_N` and `swap01_inv` are proved by `cases bool:` on the guards. The habeas quid premise `t N` is what makes `t = 0` grounded and the case split available.

5.2 Concrete definitions

Every function the locales left abstract is introduced in a single `axiomatization` block whose axioms are their recursive `:=` equations. This is the definitional mechanism of §2.8 applied at scale, and it is why a self-referential evaluator can be written down at all: no termination obligation is incurred when the definition is made.

```
axiomatization
  eval               :: "tm ⇒ asn ⇒ val"                              and
  eval_fuel          :: "num ⇒ tm ⇒ asn ⇒ val" and
  subst_T            :: "tm ⇒ tm ⇒ tm ⇒ tm"                           and
  subst_F            :: "fm ⇒ tm ⇒ tm ⇒ fm"                           and
  subst_body         :: "tm ⇒ tm ⇒ tm ⇒ tm"                           and
  check_template     :: "fm ⇒ fm ⇒ tm ⇒ tm ⇒ fm ⇒ tm ⇒ o"        and
  rep_vars_T         :: "tm ⇒ tm ⇒ tm"                                and
  rep_vars_F         :: "fm ⇒ tm ⇒ fm"                                and
  find_phi           :: "jdg ⇒ tm ⇒ tm ⇒ pf ⇒ o"                     and
  find_eq            :: "jdg ⇒ pf ⇒ pf ⇒ o"                           and
  check_subst        :: "jdg ⇒ pf ⇒ o"                                and
  check_eq_rules     :: "hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o"  and
  check_neq_rules    :: "hyp ⇒ tm ⇒ tm ⇒ tmtag ⇒ tmtag ⇒ pf ⇒ o"  and
  check_ind_template :: "fm ⇒ fm ⇒ tm ⇒ hyp ⇒ fm ⇒ tm ⇒ pf ⇒ o"  and
```

```
find_ind_base      :: "jdg ⇒ tm ⇒ pf ⇒ pf ⇒ o"                  and
check_ind          :: "jdg ⇒ pf ⇒ o"                             and
find_cut           :: "jdg ⇒ pf ⇒ pf ⇒ o"                        and
check_cut          :: "jdg ⇒ pf ⇒ o"                             and
find_struct        :: "jdg ⇒ hyp ⇒ pf ⇒ o"                       and
check_struct       :: "jdg ⇒ pf ⇒ o"                             and
app_try            :: "jdg ⇒ num ⇒ num ⇒ num ⇒ pf ⇒ o"          and
app_y              :: "jdg ⇒ num ⇒ num ⇒ num ⇒ pf ⇒ o"          and
app_x              :: "jdg ⇒ num ⇒ num ⇒ pf ⇒ o"                and
app_d              :: "jdg ⇒ num ⇒ pf ⇒ o"                       and
check_app          :: "jdg ⇒ pf ⇒ o"                             and
valid_step         :: "jdg ⇒ pf ⇒ o"                             and
check_list         :: "pf ⇒ o"                                   and
is_valid_proof     :: "pf ⇒ jdg ⇒ o"                             and
fresh_T            :: "num ⇒ tm ⇒ o"                             and
fresh_F            :: "num ⇒ fm ⇒ o"                             and
fresh_H            :: "num ⇒ hyp ⇒ o"                            and
dfn_is             :: "dfn ⇒ num ⇒ tm ⇒ o"                       and
asn_put            :: "asn ⇒ num ⇒ val ⇒ asn"
```

Thirty-three constants: the two evaluators, the three substitutions, the freshness predicates, the assignment update, and the checker hierarchy down to `is_valid_proof`. Their equations repeat the locale assumptions verbatim, for instance the direct evaluator

```
eval_def: "eval t A :=
  if tag_T t = T_VAR then nth (load_T t) A
  else if tag_T t = T_ZERO then 0
  else if tag_T t = T_SUC then S(eval (load_T t) A)
  else if tag_T t = T_PRED then P(eval (load_T t) A)
  else if tag_T t = T_IFZ then
    (if eval (cpx (load_T t)) A = 0
       then eval (cpx (cpy (load_T t))) A
       else eval (cpy (cpy (load_T t))) A)
  else
    (if eval (cpx (cpy (load_T t))) A = eval (cpx (cpy (load_T t))) A
     then (if eval (cpy (cpy (load_T t))) A = eval (cpy (cpy (load_T t))) A
           then eval (nth (cpx (load_T t)) dfns)
                  ((eval (cpx (cpy (load_T t))) A)▹ ((eval (cpy (cpy (load_T t))) A) ▹ Nil))
           else 0)
     else 0)" and
```

and the assignment update

```
asn_put_def: "asn_put A i v :=
    if i = 0 then
      if A = Nil then v ▹ Nil
      else v ▹ list_tl A
    else if A = Nil then 0 ▹ asn_put Nil (i - 1) v
    else list_hd A ▹ asn_put (list_tl A) (i - 1) v" and
```

The background definition list stays opaque. Only its termination is assumed:

```
axiomatization dfns :: "dfn" where dfns_N: "dfns N"
```

so the result holds for every terminating definition list, including lists whose definitions diverge on all arguments.

5.3 Discharging the assumptions

What remains is the arithmetic that the locale structure confined to this one file.

Habeas quid. Each of the six encoder operations preserves termination, by the corresponding pairing lemma.

```
lemma pack_F_N:
  assumes t: "t N" and L: "L N" shows "pack_F t L N"
  unfolding pack_F_def by (rule cpair_terminates[OF t L])

lemma tag_F_N:
  assumes f: "f N" shows "tag_F f N"
  unfolding tag_F_def by (rule cpx_terminates[OF f])

lemma load_F_N:
  assumes f: "f N" shows "load_F f N"
  unfolding load_F_def by (rule cpy_terminates[OF f])

lemma pack_T_N:
  assumes t: "t N" and L: "L N" shows "pack_T t L N"
  unfolding pack_T_def by (rule cpair_terminates[OF swap01_N[OF t] L])

lemma tag_T_N:
  assumes t: "t N" shows "tag_T t N"
  unfolding tag_T_def by (rule swap01_N[OF cpx_terminates[OF t]])

lemma load_T_N:
  assumes t: "t N" shows "load_T t N"
  unfolding load_T_def by (rule cpy_terminates[OF t])
```

Round-trips. Injectivity is the projection equations of §2.11 composed with `swap01_inv`. The retraction property is `cpair_reconstr`, that a number is the pair of its projections.

```
lemma tag_pack_F:
  assumes t: "t N" and L: "L N" shows "tag_F (pack_F t L) = t"
  unfolding tag_F_def pack_F_def by (rule cpx_proj[OF t L])

lemma load_pack_F:
  assumes t: "t N" and L: "L N" shows "load_F (pack_F t L) = L"
  unfolding load_F_def pack_F_def by (rule cpy_proj[OF t L])

lemma tag_pack_T:
  assumes t: "t N" and L: "L N" shows "tag_T (pack_T t L) = t"
  unfolding tag_T_def pack_T_def
  using t by (simp add: cpx_proj[OF swap01_N[OF t] L] swap01_inv[OF t])

lemma load_pack_T:
  assumes t: "t N" and L: "L N" shows "load_T (pack_T t L) = L"
  unfolding load_T_def pack_T_def by (rule cpy_proj[OF swap01_N[OF t] L])

lemma pack_tag_F:
```

Structural decrease. Both payload extractors are `cpy`, so the assumption reduces to strict decrease of the second projection on nonzero arguments, which is already available in `GD.thy`. The base case is `swap01 (cpx 0) = swap01 0 = 1`.

```
lemma decrease_F:
  assumes f: "f N" and nz: "f ≠ 0" shows "load_F f < f = 1"
  unfolding load_F_def using f nz by simp

lemma decrease_T:
  assumes t: "t N" and nz: "t ≠ 0" shows "load_T t < t = 1"
```

```
  unfolding load_T_def using t nz by simp

lemma tag_T_zero: "tag_T 0 = T_ZERO"
  unfolding tag_T_def by simp
```

Monotonicity. This is the assumption that needs work. The chain begins with `pair_step`, that a single successor step in the second component does not decrease the pair,

```
pair_step: ⟦a N; k N⟧ ⟹ ⟨a, k⟩ ≤ ⟨a, S k⟩ = 1,
```

which follows from `cpair_suc` and repeated use of `leq_monotone_add_r`. `pair_mono_2` closes it under induction on the second component, and the two interface obligations follow.

```
lemma mono_pack_T:
  assumes tg: "tg N" and x: "x N" and y: "y N" and h: "x ≤ y = 1"
  shows "pack_T tg x ≤ pack_T tg y = 1"
  unfolding pack_T_def by (rule pair_mono_2[OF swap01_N[OF tg] x y h])

lemma mono_pack_F:
  assumes tg: "tg N" and x: "x N" and y: "y N" and h: "x ≤ y = 1"
  shows "pack_F tg x ≤ pack_F tg y = 1"
  unfolding pack_F_def by (rule pair_mono_2[OF tg x y h])
```

The induction in `pair_mono_2` is a good miniature of the style of the whole development. The statement being inducted on is the object-level implication `(x ≤ y = 1) ⟶ (⟨a, x⟩ ≤ ⟨a, y⟩ = 1)`, every `implI` step first discharges a groundedness obligation through `eqBool`, and the case analysis in the step case goes through `cases_bool`.

5.4 Instantiation and the theorem

With the assumptions available the locale is interpreted. The interpretation supplies, in order, the six encoder operations, the three freshness predicates, the definition list and its membership test, the step-indexed evaluator, the three substitutions and the assignment update, and the checker hierarchy.

```
interpretation conc2: bga_fuller
  tag_T load_T pack_T tag_F load_F pack_F fresh_T fresh_F fresh_H dfns dfn_is
  eval_fuel
  subst_T subst_F subst_body asn_put
  check_template rep_vars_T rep_vars_F find_phi find_eq check_subst
  check_eq_rules check_neq_rules
  check_ind_template find_ind_base check_ind
  find_cut check_cut find_struct check_struct
  app_try app_y app_x app_d check_app
  valid_step check_list is_valid_proof
  apply unfold_locales
  apply (fact dfns_N pack_F_N tag_F_N load_F_N pack_T_N tag_T_N load_T_N
              tag_pack_F load_pack_F tag_pack_T load_pack_T pack_tag_F pack_tag_T
              decrease_F decrease_T tag_T_zero mono_pack_T mono_pack_F
              fresh_T_def fresh_F_def fresh_H_def dfn_is_def eval_fuel_def
              subst_T_def subst_F_def subst_body_def asn_put_def
              check_template_def rep_vars_T_def rep_vars_F_def find_phi_def
              find_eq_def check_subst_def check_eq_rules_def check_neq_rules_def
              check_ind_template_def find_ind_base_def check_ind_def
              find_cut_def check_cut_def find_struct_def check_struct_def
              app_try_def app_y_def app_x_def app_d_def check_app_def
              valid_step_def check_list_def is_valid_proof_def)+
  done
```

Every goal generated by `unfold_locales` is closed by `fact`: the structural obligations by the lemmas above, and the definitional obligations by the `:=` axioms of the `axiomatization` block, which are literally the equations the locale assumes.

```
theorem BGA_syntactically_consistent:
  assumes "a N" and "b N" and "p1 N" and "p2 N"
  shows "¬ (is_valid_proof p1 ⟨Nil, conc2.mk_eq a b⟩ ∧
             is_valid_proof p2 ⟨Nil, conc2.mk_neq a b⟩)"
  using assms by (rule conc2.syntactically_consistent)
```

Read out, the theorem says that for all terminating a, b, p_1 and p_2, it is not the case that p_1 codes a BGA proof of $\ulcorner a = b \urcorner$ from no hypotheses while p_2 codes a BGA proof of $\ulcorner a \neq b \urcorner$ from no hypotheses. Every symbol in that statement, the quantifiers, the negation, the conjunction and the proof checker, belongs to QGA.

6 Reduction of QGA to BGA

§4 proves BGA consistent inside QGA. To turn that into a self-consistency result, QGA itself has to be reduced to BGA, so that the system doing the proving is no stronger than the system being proved consistent. This section is to be written.

7 Conclusion

This development carries out a consistency argument for the encoded BGA proof system in QGA, implemented as an extension of Isabelle/Pure. The theorem `BGA_syntactically_consistent` states that no two proof codes are accepted, from empty hypotheses, for an equation and its corresponding disequality. The argument proves the checker sound and uses determinism of evaluation to rule out such a pair.

Two features make the proof manageable. The abstract encoding interface separates the soundness argument from the arithmetic of a particular Gödel numbering; the Cantor-pairing construction then supplies that interface. Evaluation at a fixed fuel budget makes the satisfaction premises used in the proof-list induction decidable, allowing the argument to meet QGA's habeas quid requirements.

An attempt to extend the construction to QGA itself is recorded in Appendix A for future collaborators.

References

[1] Sascha Kehrli. *Formalizing Grounded Arithmetic atop Isabelle/Pure*. BSc thesis, Distributed Computing Group, ETH Zürich, 30 September 2025.

[2] Bryan Ford. *Computable Quantification in Reflective Grounded Arithmetic*. arXiv:2607.25533, 2026. `https://arxiv.org/abs/2607.25533`.

A An unfinished attempt at QGA self-consistency

This appendix records an attempt to prove QGA's consistency within QGA itself. The attempt was not completed and has been abandoned. I retain this account in case its ideas or difficulties are useful to future collaborators. The experimental files `pure/QGA_on_GA.thy` and `pure/qga_encode.thy` remain in the repository, but they contain unproved lemmas and do not establish QGA self-consistency.

A.1 The proposed semantics

The aim was to repeat the BGA construction with encoded QGA formulas and proofs. A formula could evaluate to true, to false, or to neither; the last case would represent an ungrounded formula. Equality would compare terminating terms, negation would exchange truth and falsity, and disjunction would be true when either disjunct was true and false when both were false.

Quantifiers were the main difficulty. Reading a universal as true when every numeral instance is true leaves infinitely many instances to check, even with a fixed evaluation budget. The alternative explored was to search for a proof of the body for an arbitrary natural number, and for a refutation of an instance to establish falsity. This was motivated by reflective grounded arithmetic [2]. Bounding proof search would permit finite checks, but relating their results to truth would require further proofs: a certificate cannot be treated as sound merely because the checker accepts it.

A.2 Ideas retained from the attempt

The BGA proof suggested keeping hypothesis satisfaction decidable at a fixed budget, while allowing a less restrictive predicate for conclusions. The attempt explored an additional "witnessed" semantics, in which a quantifier certificate was accompanied by the semantic facts it was meant to establish. Separate searches for truth and falsity were also considered, to avoid making one search take priority over the other.

Another proposal allowed a quantifier certificate to retain closed hypotheses, provided their satisfaction was checked within a recorded budget. The intention was to reuse a derivation already present in the proof being checked, rather than first turn every satisfied hypothesis into a proof from no hypotheses.

These were proposed ways to organize the argument, not established solutions. In particular, the necessary substitution and soundness arguments were not completed.